

Bộ giáo dục và đào tạo
Trường Đại học Sài Gòn

--C3 80--



Báo cáo: Đồ án giữa kì
Môn: Công Nghệ Lập Trình Hiện Đại
Đề tài: Khóa Học UDEMY LLM

Nhóm sinh viên thực hiện:

MSSV	Họ và tên SV
3121411226	Trần Quang Vinh
3121411060	Nguyễn Trí Đức
3121411045	Đặng Nguyễn Quốc Dương
3121411100	Phạm Quang Khiêm

Giáo viên hướng dẫn: **TS.Đỗ Như Tài**

TP.HCM, Tháng 5, năm 2025

MỤC LỤC

CHƯƠNG 1: BUILD YOUR FIRST LLM PRODUCT	1
1.1.MÔ HÌNH NGÔN NGỮ LỚN (LLM - LARGE LANGUAGE MODEL)	1
1.2.CÁC HÌNH THỨC TRIỂN KHAI LLM.....	3
1.3.CÁC THÀNH PHẦN CỦA ỨNG DỤNG LLM CƠ BẢN	3
1.4.TẠO HÀM HỎI ĐÁP	4
1.5.DEMO CODE TUẦN 1	5
CHƯƠNG 2: BUILD A MULTI-MODAL CHATBOT (LLMS, GRADIO UI, VÀ AGENTS IN ACTION)	8
2.1.GIỚI THIỆU VỀ CHATBOT ĐA PHƯƠNG THỨC (MULTI-MODAL CHATBOT	8
2.2.ÔN LẠI VỀ LLM TRONG XÂY DỰNG CHATBOT	9
2.3.XÂY DỰNG GIAO DIỆN NGƯỜI DÙNG BẰNG GRADIO	9
2.4.AGENTS VÀ KỸ THUẬT AGENTIC WORKFLOW	9
2.5.THỰC HÀNH: TẠO CHATBOT ĐƠN GIẢN.....	11
CHƯƠNG 3: XÂY DỰNG GIẢI PHÁP TỰ ĐỘNG HÓA VỚI HUGGING FACE	14
3.1.LÀM QUEN VỚI HUGGING FACE:	14
3.2.TÌM HIỂU GIẢI PHÁP TỰ ĐỘNG HÓA VỚI HUGGING FACE.....	15
3.3.QUY TRÌNH CÁC BƯỚC XÂY DỰNG GIẢI PHÁP TỰ ĐỘNG HÓA VỚI HUGGING FACE	17
3.4.MÔ HÌNH XÂY DỰNG GIẢI PHÁP TỰ ĐỘNG HÓA VỚI HUGGING FACE:	19
3.4.1.Load và chạy thử một mô hình chat open-source.....	19
3.4.2.Tạo một API nội bộ nhỏ.....	20
3.4.3. Tổng hợp giọng nói (Text-to-Speech)	22
3.4.4.Tạo hình ảnh từ văn bản (Text-to-Image generation)	23
3.4.5. Thực hiện quy trình tự động chuyển đổi nội dung một buổi họp từ file âm thanh sang biên bản họp chi tiết (meeting minutes).....	24

CHƯƠNG 4: ĐÁNH GIÁ MÔ HÌNH CHO SINH MÃ (CODE GENERATION) VÀ NHIỆM VỤ KINH DOANH (BUSINESS TASKS)	28
4.1.TẦM QUAN TRỌNG CỦA VIỆC ĐÁNH GIÁ LLM.....	28
4.2.CÁC NHÓM NHIỆM VỤ CẦN ĐÁNH GIÁ	28
4.3.CÁC MÔ HÌNH THAM GIA “SHOWDOWN” (SO TÀI).....	29
4.4.CÁCH TIẾN HÀNH ĐÁNH GIÁ MÔ HÌNH (LLM EVALUATION)	30
4.5.CODE DEMO SO SÁNH 2 MÔ HÌNH GEMINI VÀ OLLAMA MISTRAL VỀ BUSINESS TASKS VÀ CODE GENERATION	31
4.5.1. <i>Kết quả khi so sánh Code Generation:</i>	32
4.5.2. <i>Kết quả khi so sánh Business Tasks</i>	33
CHƯƠNG 5: XÂY DỰNG GIẢI PHÁP NÂNG CAO VỚI VECTOR EMBEDDINGS & LANGCHAIN	34
5.1.RAG LÀ GÌ? (RETRIEVAL-AUGMENTED GENERATION).....	34
5.2.KIẾN TRÚC CƠ BẢN CỦA MỘT HỆ THỐNG RAG.....	36
5.3.CÁC THÀNH PHẦN QUAN TRỌNG TRONG RAG	37
5.4.TÓM TẮT LANGCHAIN (FRAMEWORK CHO RAG).....	37
5.5.NHỮNG LƯU Ý KHI XÂY DỰNG RAG CHẤT LƯỢNG CAO.....	38
5.6. DEMO GIẢI PHÁP NÂNG CAO VỚI VECTOR EMBEDDINGS & LANGCHAIN	39
CHƯƠNG 6: TÍNH CHỈNH CÁC MÔ HÌNH LLM TIÊN TIẾN BẰNG KỸ THUẬT LORA VÀ QLoRA	43
6.1. <i>Vì sao cần FINE-TUNING LLM?</i>	43
6.2.LoRA và QLoRA LÀ GÌ?	44
6.3.KIẾN TRÚC FINE-TUNING VỚI LoRA / QLoRA.....	44
6.4.CÁC THÀNH PHẦN TRONG QUY TRÌNH FINE-TUNING	45
6.5.CÁC MẸO FINE-TUNING THỰC TẾ	46
6.6.DEMO CODE MÔ HÌNH LoRA VÀ QLoRA	46

CHƯƠNG 7: HUẤN LUYỆN MÔ HÌNH MÃ NGUỒN MỞ ĐỂ CẠNH TRANH VỚI FRONTIER MODELS TRONG DỰ ĐOÁN GIÁ 49

7.1. BỐI CẢNH VÀ MỤC TIÊU	49
7.2. CÁC BƯỚC TỔNG THỂ CỦA DỰ ÁN.....	50
7.2.1. Bắt đầu:	50
7.2.2. Cấu hình mô hình	51
7.2.3. Tải tập dữ liệu, mô hình và bộ mã hóa.....	52
7.2.4. Cấu hình lượng tử hóa 4 bit	53
7.2.5. Tải mô hình Llama 2.....	55
7.2.6. Các thông số đào tạo	56
7.2.7. Đánh giá	59

CHƯƠNG 8: XÂY DỰNG HỆ THỐNG TỰ ĐỘNG HÓA VỚI CÁC TÁC NHÂN (AGENTS) HỢP TÁC CÙNG CÁC MÔ HÌNH LLM 62

8.1. GIỚI THIỆU VỀ MULTI-AGENT SYSTEMS (MAS)	62
8.1.1 Agent Orchestration (Điều phối tác nhân):.....	63
8.1.2 Agents (Tác nhân):	63
8.1.3 Human in the middle (Người dùng ở giữa):.....	63
8.1.4. Quy trình hoạt động:	63
8.2. TẠI SAO CẦN MULTI-AGENT SYSTEMS?.....	64
8.3. CÁC LOẠI TÁC NHÂN TRONG MAS	64
8.4. XÂY DỰNG HỆ THỐNG TÁC NHÂN TỰ ĐỘNG (AUTONOMOUS MULTI-AGENT SYSTEM).....	65
8.4.1 Kiến trúc chung	65
8.4.2 Mô hình Collaborative Multi-Agent System	65
8.5. ỨNG DỤNG THỰC TẾ CỦA MULTI-AGENT SYSTEMS	66
8.6. CÔNG CỤ VÀ FRAMEWORKS HỖ TRỢ	66

CHƯƠNG 9: TÓM TẮT NỘI DUNG HỌC ĐƯỢC 74

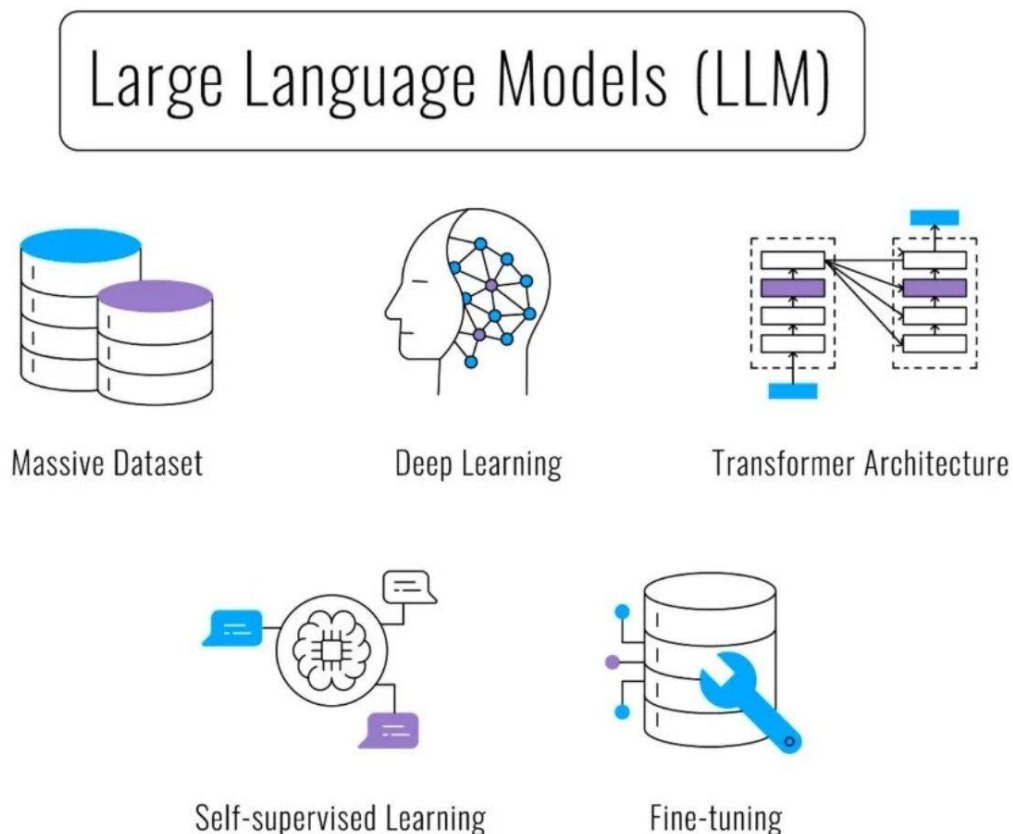
Chương 1: Build Your First LLM Product

Mục tiêu tuần 1

- Làm quen với cách tích hợp mô hình ngôn ngữ lớn (LLM) vào một ứng dụng thực tế.
- Hiểu sự khác nhau giữa mô hình cloud (như Gemini của Google) và mô hình local (như LLaMA chạy qua Ollama).
- Xây dựng một chương trình đơn giản cho phép người dùng đặt câu hỏi và nhận câu trả lời từ LLM.

Lý thuyết nền tảng

1.1.Mô hình ngôn ngữ lớn (LLM - Large Language Model)



Hình 1: LLM được đào tạo như thế nào

Dựa vào hình 1 ta có:

❖ **Massive Dataset (Tập dữ liệu khổng lồ)**

LLM được huấn luyện trên hàng tỷ từ từ nhiều nguồn như sách, website, mạng xã hội, v.v. Dữ liệu càng lớn, mô hình càng học được nhiều kiến thức và ngữ cảnh.

❖ **Deep Learning (Học sâu)**

LLM sử dụng mạng nơ-ron nhiều lớp (deep neural networks) để học các mẫu ngôn ngữ phức tạp, giúp mô hình hiểu và tạo ra văn bản tự nhiên.

❖ **Transformer Architecture (Kiến trúc Transformer)**

Transformer là kiến trúc nền tảng đứng sau hầu hết các LLM hiện nay (như GPT, BERT, Gemini, Claude...). Nó giúp mô hình xử lý văn bản theo ngữ cảnh một cách hiệu quả và song song.

❖ **Self-supervised Learning (Học tự giám sát)**

Mô hình được huấn luyện mà không cần nhãn thủ công — ví dụ: đoán từ tiếp theo trong câu. Đây là phương pháp cực kỳ hiệu quả cho việc huấn luyện trên dữ liệu lớn.

❖ **Fine-tuning (Điều chỉnh tinh)**

Sau khi huấn luyện ban đầu, LLM có thể được tinh chỉnh (fine-tune) trên các tập dữ liệu nhỏ hơn, chuyên biệt cho một nhiệm vụ cụ thể như chatbot, phân tích cảm xúc, hoặc viết code.

1.2.Các hình thức triển khai LLM

Hình thức	Mô tả	Ví dụ
Cloud-based	Mô hình chạy trên hạ tầng đám mây của nhà cung cấp. Truy cập qua API.	OpenAI GPT, Gemini, Claude
Local-based	Mô hình chạy trực tiếp trên máy cá nhân hoặc server riêng.	LLaMA (qua Ollama), Mistral, Phi-2

Bảng 1: So sánh mô hình Cloud-based và Local-based

1.3.Các thành phần của ứng dụng LLM cơ bản

❖ Quản lý biến môi trường

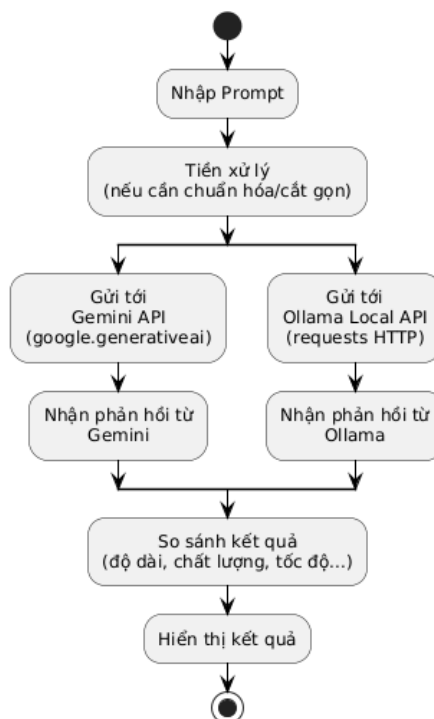
Dùng thư viện dotenv để giữ an toàn cho API key.

❖ Giao tiếp với LLM

Gemini: dùng thư viện google.generativeai để gửi yêu cầu và nhận phản hồi.

Ollama: gửi yêu cầu HTTP đến local server qua requests.

So sánh Gemini API và Ollama Local API



Hình 2: Mô hình hóa so sánh Gemini API và Olama Local API

Hình 2 là một sơ đồ khối mô tả quá trình so sánh giữa Gemini API và Ollama Local API. Dưới đây là mô tả chi tiết từng bước:

1. Bắt đầu : Quy trình bắt đầu từ đây.
2. Nhập Prompt: Người dùng nhập một câu lệnh (prompt).
3. Tiền xử lý (nếu cần chuẩn hóa/cắt gọn): Prompt có thể được tiền xử lý, ví dụ như chuẩn hóa hoặc rút gọn, nếu cần thiết.
4. Gửi tới Gemini API (google.generativeai): Prompt sau đó được gửi đến Gemini API của Google (thông qua thư viện google.generativeai).
5. Gửi tới Ollama Local API (requests HTTP): Đồng thời, prompt cũng được gửi đến Ollama Local API thông qua các yêu cầu HTTP (requests HTTP).
6. Nhận phản hồi từ Gemini: Hệ thống nhận phản hồi từ Gemini API.
7. Nhận phản hồi từ Ollama: Hệ thống nhận phản hồi từ Ollama Local API.
8. So sánh kết quả (độ dài, chất lượng, tốc độ...): Các phản hồi nhận được từ Gemini và Ollama sẽ được so sánh dựa trên các tiêu chí như độ dài, chất lượng, tốc độ xử lý,...
9. Hiển thị kết quả: Kết quả so sánh sẽ được hiển thị cho người dùng.
10. Kết thúc: Quy trình kết thúc.

Tóm lại, sơ đồ này minh họa một quy trình thử nghiệm hoặc so sánh hiệu suất và chất lượng giữa một API đám mây (Gemini API) và một API chạy cục bộ (Ollama Local API) đối với cùng một prompt đầu vào.

1.4.Tạo hàm hỏi đáp

Hàm ask_gemini(question) dùng Gemini để trả lời.

Hàm ask_ollama(question) dùng mô hình local như LLaMA 3 để trả lời.

1.5.Demo code tuần 1

Gửi một câu hỏi đơn giản đến cả hai mô hình và in ra kết quả.

```
load_dotenv()

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

gemini_model = genai.GenerativeModel("gemini-1.5-pro") # hoặc "gemini-pro"

def ask_gemini(question):
    response = gemini_model.generate_content(question)
    return response.text.strip()

# Constants
OLLAMA_API = "http://localhost:11434/api/chat"
HEADERS = {"Content-Type": "application/json"}
MODEL = "llama3.2"

def ask_ollama(question, model=MODEL):
    payload = {
        "model": model,
        "messages": [{"role": "user", "content": question}]
    }
    try:
        response = requests.post(OLLAMA_API, headers=HEADERS,
data=json.dumps(payload))

        if response.status_code != 200:
            return f"Ollama API Error: {response.status_code}, {response.text}"

        response_lines = response.text.splitlines()
        final_message = ""
```

```

for line in response_lines:
    try:
        response_data = json.loads(line)
        message = response_data.get("message", {}).get("content", None)
        if message:
            final_message += message
    except json.JSONDecodeError:
        continue
    return final_message if final_message else " No response content."

except requests.exceptions.RequestException as e:
    return f" Request Error: {e}"
except Exception as e:
    return f" Ollama Error: {e}"

def ask_gemini(question):
    prompt = f"Trả lời ngắn gọn bằng tiếng Việt: {question}"
    response = gemini_model.generate_content(prompt)
    print("Gemini says:\n")
    print(response.text.strip())
ask_gemini("Giải thích dòng code: yield from {book.get('author')} for book in books if
book.get('author')}")
# Test
question = "Giải thích dòng code: yield from {book.get('author')} for book in books if
book.get('author')}")
print("Ollama says:\n")
print(ask_ollama(question))

```

Output:

Gemini: giải thích nhanh, súc tích, phù hợp khi cần đọc gọn.

```
Gemini says:
Dòng code này tạo ra một generator trả về lần lượt tên của các tác giả (không trùng lặp) từ danh sách các cuốn sách ('books').

**Giải thích chi tiết:**

* 'books': Là một danh sách các từ điển, mỗi từ điển đại diện cho một cuốn sách.
* 'book.get('author')': Lấy giá trị của khóa 'author' từ mỗi cuốn sách. 'get()' được dùng để tránh lỗi nếu một cuốn sách không có khóa 'author'.
* 'if book.get('author')': Chỉ lấy những cuốn sách có trường 'author' không rỗng.
* '{... for book in books if book.get('author')}': Tạo một set comprehension, tức là một tập hợp (không chứa các giá trị trùng lặp) các tên tác giả.
* 'yield from': Lặp qua tập hợp các tên tác giả và trả về từng tên một dưới dạng một generator. Điều này có nghĩa là giá trị sẽ được trả về khi cần thiết (lazy).

Nói tóm lại, nó lấy tên các tác giả từ danh sách sách, loại bỏ trùng lặp và trả về kết quả từng phần một khi cần thiết.
```

Ollama (llama3.2): giải thích có ví dụ minh họa, chi tiết hơn, phù hợp cho người muốn thấy cách áp dụng trong code thực tế.

```
ollama says:
Dòng code bạn cung cấp sử dụng tính năng 'yield from' trong Python, giúp thực hiện một cách linh hoạt hơn khi xử lý các kết quả từ các generator hoặc các giá trị lặp.
Dưới đây là giải thích chi tiết:

- 'for book in books': Đây là vòng lặp để đi qua mỗi bản ghi ('book') trong danh sách ('books').
- 'if book.get('author')': Trong vòng lặp, chỉ khi bản ghi có giá trị của khóa 'author', thì mới thực hiện tiếp các bước sau.
- '{book.get('author') for book in books if book.get('author')}': Đây là một biểu thức sử dụng từ khóa 'for' và tạo ra một generator. Biểu thức này sẽ đi qua tất cả các 'book' trong 'books' và chỉ lấy ra giá trị 'author' nếu nó tồn tại.
- 'yield from ...': Tính năng 'yield from' được giới thiệu trong Python 3.3, giúp bạn có thể "chuyển" kết quả từ một generator sang kết quả của các generator khác.
- Tóm lại, dòng code 'yield from {book.get('author') for book in books if book.get('author')}' sẽ tạo ra một generator có giá trị bằng cách lấy giá trị của khóa 'author' từ các cuốn sách có trường 'author' không rỗng.

```python
def get_authors(books):
 yield from {book.get('author') for book in books if book.get('author')}

Sau khi khai báo hàm 'get_authors', bạn có thể sử dụng nó với vòng lặp như sau:
```python
books = [{'title': 'Book1', 'author': 'Author1'}, {'title': 'Book2', 'author': ''}, {'title': 'Book3'}]
for author in get_authors(books):
    print(author)
```
```

## Kết luận tuần 1:

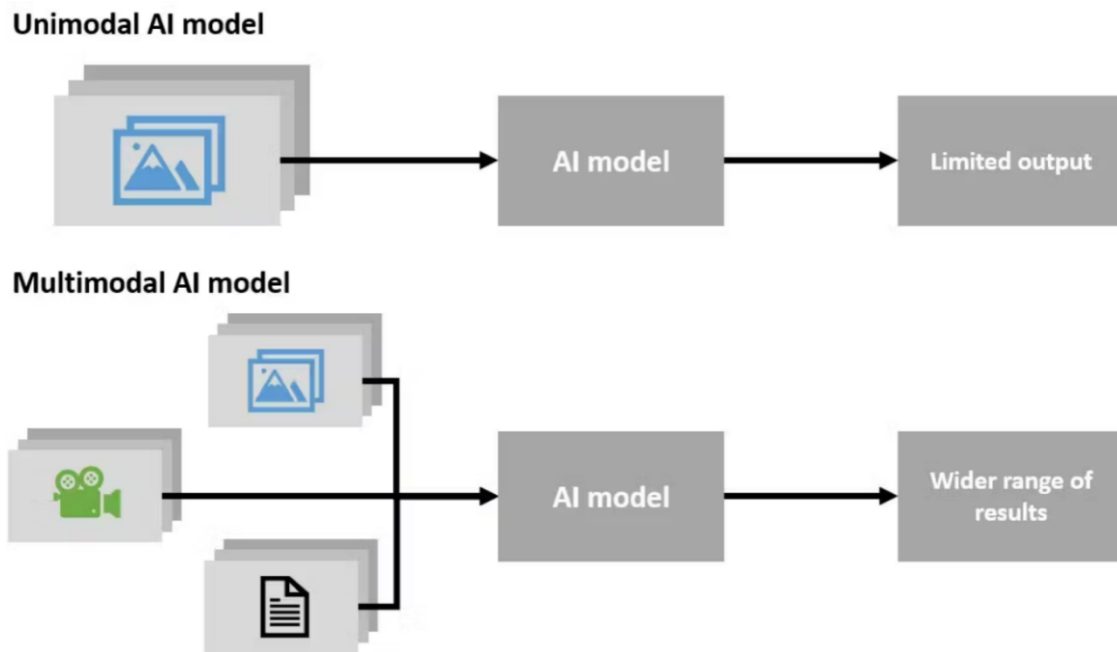
**Tích hợp LLM vào ứng dụng Python (Cloud & Local):** Có khả năng tích hợp các mô hình ngôn ngữ lớn (LLM) vào ứng dụng Python, bao gồm cả việc kết nối với mô hình chạy trên cloud qua API và triển khai các mô hình local trên máy chủ hoặc máy cá nhân.

**Gửi câu hỏi và nhận câu trả lời từ các mô hình:** Hiểu cách gửi câu hỏi văn bản đến các mô hình LLM, không phân biệt là mô hình chạy trên cloud hay local, và nhận lại câu trả lời phù hợp từ các mô hình đó.

**So sánh kết quả của các mô hình LLM khác nhau:** Biết cách đánh giá và so sánh các kết quả đầu ra (output) của các mô hình LLM khác nhau, từ đó có thể lựa chọn mô hình phù hợp cho từng yêu cầu cụ thể.

**Phát triển ứng dụng AI dựa trên LLM:** Có nền tảng vững chắc để phát triển các ứng dụng AI thông minh sử dụng LLM, bao gồm việc xây dựng các chatbot, hệ thống hỏi đáp, phân tích cảm xúc, và các ứng dụng khác phục vụ nhiều mục đích trong thực tế.

## Chương 2: Build a Multi-Modal Chatbot (LLMs, Gradio UI, và Agents in Action)



Hình 3: So sánh 2 model Unimodal AI và Multimodal AI phương pháp sử dụng mô hình

### 2.1. Giới thiệu về Chatbot Đa phương thức (Multi-modal Chatbot)

#### ❖ Unimodal AI model

- Chỉ xử lý một loại dữ liệu (ví dụ: chỉ hình ảnh).
- Đầu ra (output) bị giới hạn vì chỉ dựa trên một khía cạnh.
- Ví dụ: mô hình nhận diện ảnh chỉ có thể mô tả hình ảnh nhưng không hiểu nếu có kèm văn bản hoặc âm thanh.

#### ❖ Multimodal AI model

- Có thể xử lý nhiều loại dữ liệu cùng lúc:
  - Hình ảnh
  - Video
  - Văn bản
- Từ đó tạo ra kết quả phong phú và sâu sắc hơn.

- Ví dụ: một chatbot có thể xem ảnh sản phẩm, hiểu câu hỏi người dùng, và trả lời chính xác dựa trên cả hình ảnh và văn bản.

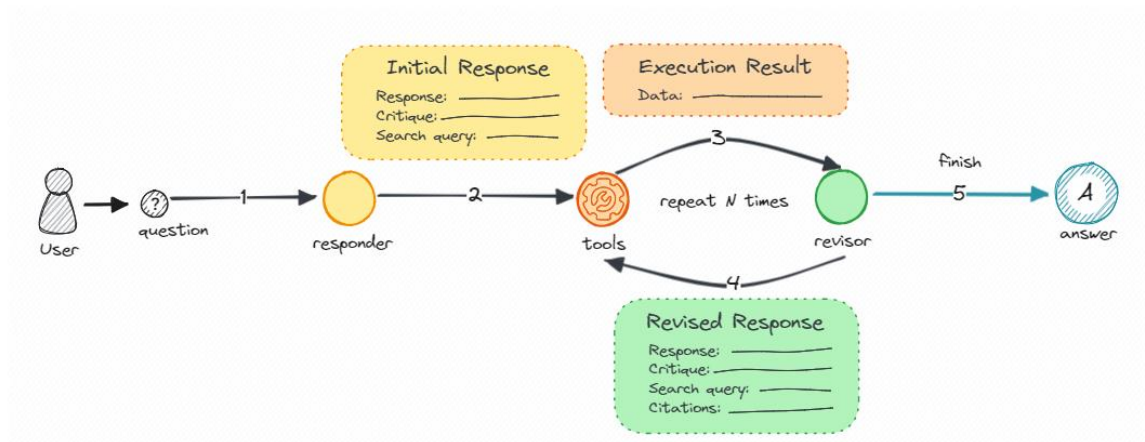
## 2.2. Ôn lại về LLM trong xây dựng chatbot

- Cách LLM tạo câu trả lời phù hợp với bối cảnh.
- Thêm "context" vào câu hỏi để tăng độ chính xác.
- Giới thiệu **multi-modal LLMs** nổi bật:
  - **GPT-4 Turbo Vision**: nhận dạng hình ảnh.
  - **Gemini (Google)**: xử lý văn bản + hình ảnh đồng thời.
  - **LLaVA, BLIP-2**: mô hình open-source nhận dạng hình ảnh.

## 2.3. Xây dựng giao diện người dùng bằng Gradio

- Gradio: Thư viện Python để nhanh chóng tạo giao diện web cho AI app.
- Các thành phần chính của Gradio:
  - Textbox, Image, Button, Chatbot.
- Cách kết nối Gradio UI với model backend.
- Demo: Tạo một chatbot đơn giản với text input và output.

## 2.4. Agents và kỹ thuật Agentic Workflow



Hình 4: Mô hình AI kết hợp giữa Tools Revisor và mô hình AI

Mô hình xử lý phản hồi nhiều bước của hệ thống AI

**Bước 1: Người dùng đặt câu hỏi**

- Người dùng gửi một câu hỏi hoặc yêu cầu vào hệ thống.

**Bước 2: Responder (bộ phản hồi ban đầu)**

- Mô hình AI đầu tiên tạo ra một phản hồi sơ bộ, bao gồm:
  - Response: câu trả lời ban đầu.
  - Critique: tự đánh giá, phân tích những điểm chưa ổn.
  - Search query: nếu cần, đề xuất tìm kiếm thêm dữ liệu.

**Bước 3: Tools (công cụ thực thi)**

- Các công cụ hoặc API được dùng để thực thi hành động dựa trên phản hồi (ví dụ: thực hiện tìm kiếm web, chạy mã, truy xuất dữ liệu).
- Kết quả thu được gọi là Execution Result (Dữ liệu thực thi).

**Bước 4: Revisor (trình chỉnh sửa)**

- Dựa trên dữ liệu từ bước 3, hệ thống tạo ra Revised Response (Phản hồi chỉnh sửa):
  - Response: câu trả lời mới.
  - Critique: đánh giá lại.
  - Search query: cập nhật nếu cần.
  - Citations: thêm trích dẫn hoặc nguồn tham khảo.

**Bước 5: Quá trình lặp lại**

- Bước 3 và 4 được lặp lại N lần để cải thiện chất lượng câu trả lời.

**Bước 6: Trả kết quả cuối cùng**

- Sau khi các bước lặp hoàn tất hoặc đạt độ tin cậy mong muốn, hệ thống trả lại câu trả lời cuối cùng cho người dùng.

## 2.5.Thực hành: Tạo Chatbot đơn giản

- Bước 1: Gọi LLM (như Gemini) để chat văn bản.
- Bước 2: Thêm upload hình ảnh (cho LLM xử lý).
- Bước 3: Dùng Gradio xây dựng giao diện:
  - Textbox cho nhập câu hỏi.
  - Image upload (tùy chọn).
  - Chat window hiển thị hội thoại.

```
Hàm xử lý câu hỏi văn bản (Gemini)
def ask_gemini(text):
 model = genai.GenerativeModel('gemini-2.0-flash')
 response = model.generate_content(text)
 return response.text.strip()

Hàm xử lý hình ảnh bằng Ollama (model llava)
def describe_image(image_np):
 # Chuyển numpy array → PIL Image → bytes
 img = Image.fromarray(image_np)
 img_buffer = BytesIO()
 img.save(img_buffer, format='JPEG')
 img_bytes = img_buffer.getvalue()

 # Encode ảnh thành base64
 img_base64 = base64.b64encode(img_bytes).decode('utf-8')

 # Gửi request đến Ollama API (model llava, dùng streaming)
 payload = {
 "model": "llava:7b",
 "messages": [
 {"role": "user", "content": "Mô tả bức ảnh này bằng tiếng Việt. Đây có phải là giày thể thao phù hợp để đá bóng không?"}
],
 "images": [img_base64],
 "stream": True
 }
 try:
 response = requests.post("http://localhost:11434/api/chat", json=payload,
stream=True)
 response.raise_for_status()
```

```

Xử lý từng dòng phản hồi mã JSON
content = ""
for line in response.iter_lines():
 if line:
 try:
 part = json.loads(line.decode("utf-8"))
 content += part.get("message", {}).get("content", "")
 except json.JSONDecodeError as je:
 print(" ⚠️ Lỗi giải mã JSON:", je)

 return content.strip() if content else " ⚠️ Không nhận được phản hồi từ mô hình Ollama."
except Exception as e:
 return f"Lỗi khi gọi Ollama: {e}"

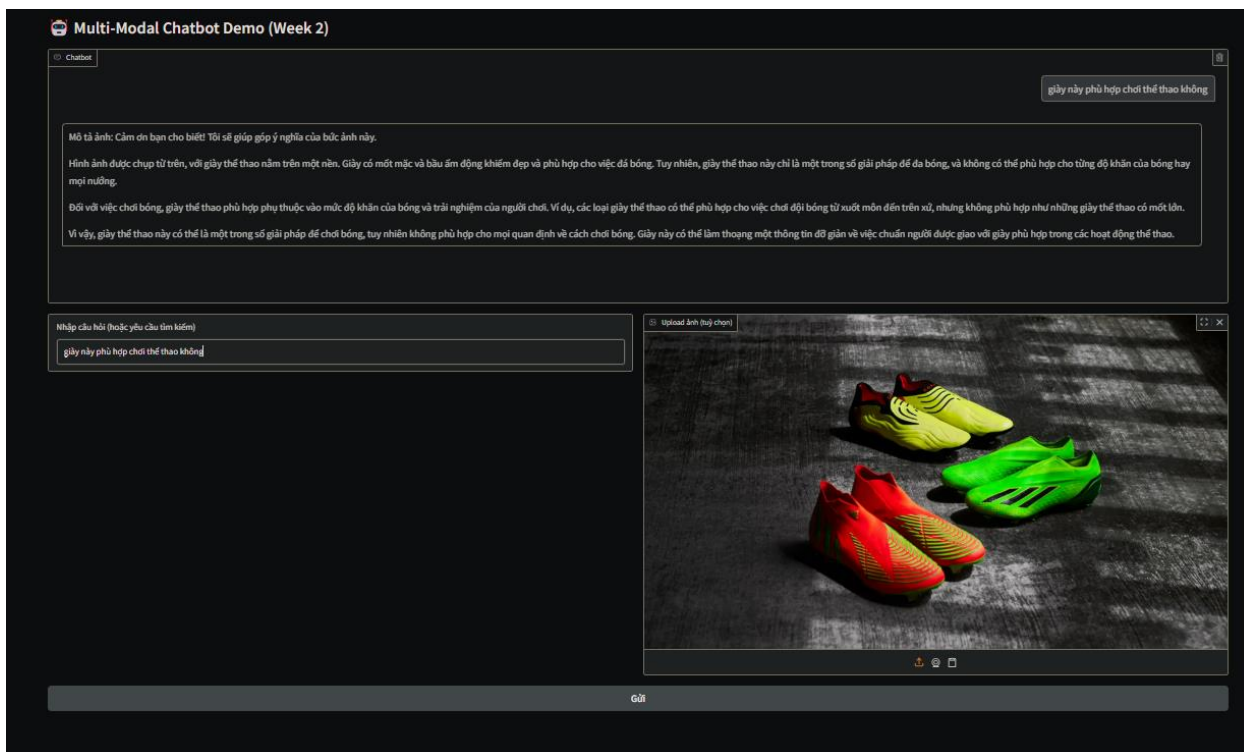
(Nâng cao) Hàm agent giả lập tìm kiếm Google
def agent_search(query):
 return f"[Agent] Tôi đã tìm kiếm Google và thấy thông tin: '{query}' - Kết quả mô phỏng."

Hàm chính xử lý input
def chatbot_response(message, image=None):
 if image is not None:
 description = describe_image(image)
 return f"Mô tả ảnh: {description}"
 elif "tìm kiếm" in message.lower():
 search_result = agent_search(message)
 return search_result
 else:
 answer = ask_gemini(message)
 return answer

```



Giải thích code: Dùng germini để trả lời câu hỏi đầu tiên, tiếp theo dùng mô hình Ollama để phân tích hình ảnh từ đó áp dụng Revisor (trình chỉnh sửa): để lặp lại kết quả câu trả lời đến khi câu trả lời đúng.



Hình ảnh 5: Mô hình demo chatbot xử lý hình ảnh và văn bản cùng lúc

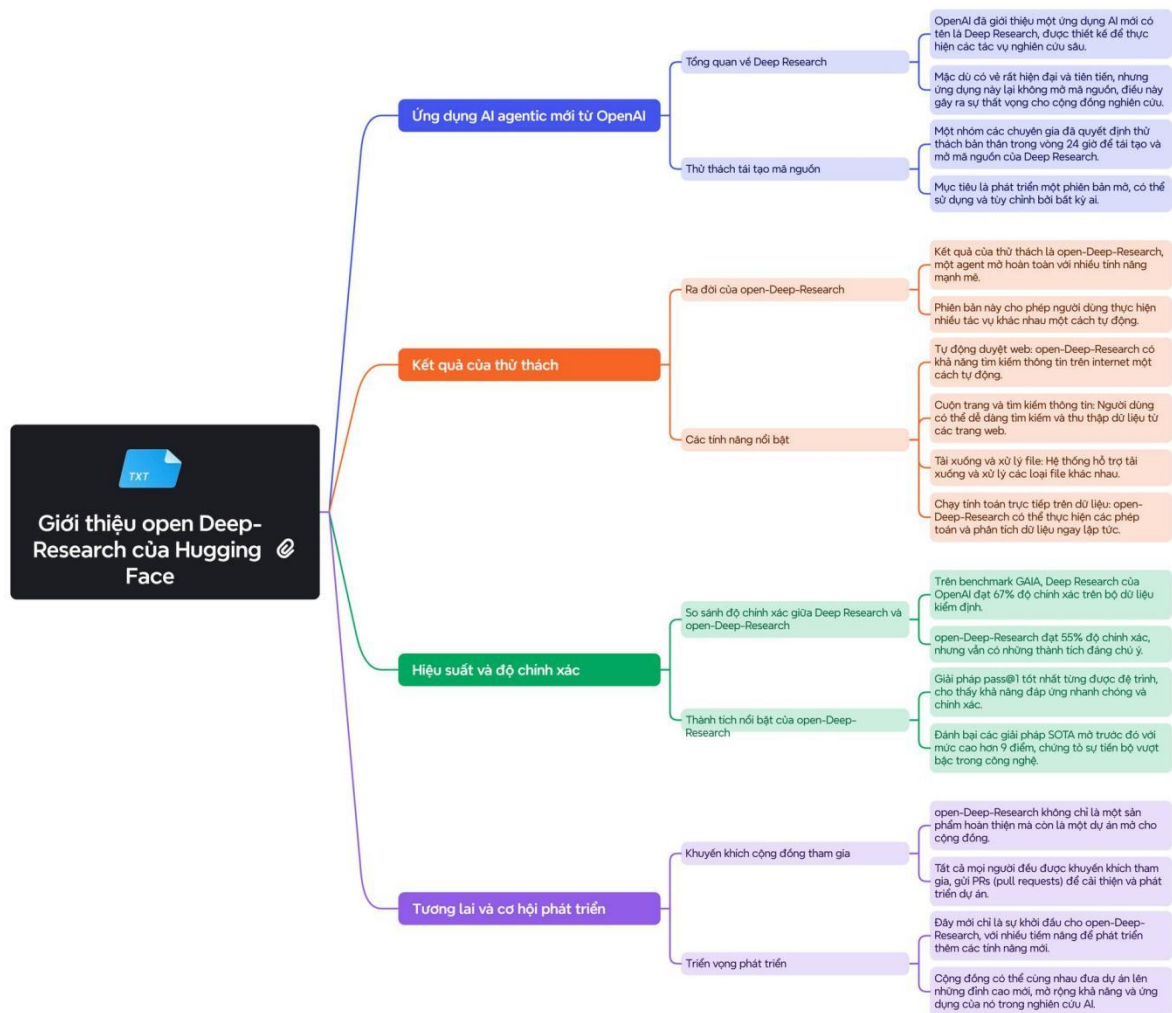
## Chương 3: Xây dựng Giải pháp Tự động hóa với Hugging Face

### 3.1. Làm quen với Hugging Face:

Để bắt đầu làm việc với Hugging Face, cần làm quen với một số thành phần chính:

- Hugging Face Hub: Đây là trái tim của hệ sinh thái. Bạn có thể truy cập nó qua trang web [huggingface.co](https://huggingface.co). Tại đây, bạn sẽ tìm thấy:
  - Models: Hàng ngàn mô hình cho đủ loại tác vụ và ngôn ngữ. Mỗi mô hình có một trang riêng hiển thị thông tin, giấy phép, hướng dẫn sử dụng và thậm chí là demo trực tiếp.
  - Datasets: Các bộ dữ liệu công khai để huấn luyện hoặc đánh giá mô hình.
  - Spaces: Các ứng dụng demo (thường sử dụng Gradio hoặc Streamlit) cho phép bạn thử nghiệm các mô hình trực tiếp trên trình duyệt.
  - Docs: Tài liệu hướng dẫn chi tiết cho tất cả các thư viện.
- Thư viện transformers: Đây là thư viện Python chính để làm việc với các mô hình. Nó cung cấp các lớp (classes) và hàm (functions) để tải mô hình, tokenizer và thực hiện inference (dự đoán) hoặc huấn luyện.
- Thư viện tokenizers: Khi làm việc với văn bản, bạn cần chuyển đổi văn bản thành các định dạng số mà mô hình có thể hiểu được (tokens). Thư viện tokenizers cung cấp các công cụ hiệu quả và linh hoạt để thực hiện việc này, thường đi kèm với mô hình văn bản tương ứng.
- Thư viện datasets: Giúp bạn tải, xử lý và quản lý các bộ dữ liệu lớn một cách hiệu quả. Rất hữu ích khi bạn cần tinh chỉnh mô hình.

### 3.2. Tìm hiểu giải pháp Tự động hóa với Hugging Face



Hình 6: Giới thiệu open DeepResearch củaHugging Face

Hình 6 giới thiệu open DeepResearch củaHugging Face bao gồm các chức năng sau:

### **Ứng dụng AI agentic mới từ OpenAI**

#### ➤ Tổng quan về Deep Research

- Nêu bối cảnh về làn sóng AI agentic (AI tự chủ hành động).
- Deep Research là một thử nghiệm mở cho phép kiểm tra năng lực của các AI agent trong các tác vụ phức tạp.
- Mục tiêu là kiểm tra khả năng duy trì mục tiêu, truy xuất thông tin, và tự tổ chức hành vi.

#### ➤ Thử thách từ tạo ra nguồn

- Tạo thử thách mang tính mở, nơi AI phải chủ động tìm thông tin.
- Các agent không thể chỉ phản hồi mà phải “tự khám phá và tổ chức hành động”.

### **Kết quả của thử thách**

#### ➤ Phản hồi của open-Deep-Research

- Có sự hưởng ứng và tham gia tích cực từ cộng đồng.
- Nhiều người dùng hào hứng thử sức AI trong môi trường mô phỏng nghiên cứu.

#### ➤ Các tính năng nổi bật

- Giao diện đơn giản, trực quan.
- AI có thể truy cập và tổng hợp tài liệu trên Hugging Face hoặc từ Internet.
- Agent hoạt động theo logic “task + subtask”, thực hiện nhiều bước liên tiếp để hoàn thành mục tiêu.

### **Hiệu suất và độ chính xác**

#### ➤ So sánh với các hệ thống khác

- Deep Research được benchmark với GPT-4, Claude, v.v.
- Kết quả cho thấy AI đạt ~55% độ chính xác khi không được can thiệp bởi con người.

#### ➤ Thành tích nổi bật

- Có những tác vụ mà AI hoàn thành tốt hơn con người.
- Deep Research hỗ trợ đánh giá khách quan hiệu suất và độ chính xác của agent.

## Tương lai và cơ hội phát triển

### ► Khuyến khích cộng đồng tham gia

- Hugging Face mở nền tảng này để cộng đồng đóng góp tác vụ mới.
- Kỳ vọng tạo ra hệ sinh thái open-source dành cho AI agentic.

### ► Tiềm năng phát triển

- Có thể mở rộng để hỗ trợ nghiên cứu học thuật, khoa học, y tế, và các lĩnh vực khác.
- Là bước đệm cho các hệ thống AI có khả năng tự nghiên cứu trong tương lai.

## 3.3. Quy trình các bước xây dựng giải pháp tự động hóa với Hugging Face



Hình ảnh 7: Quy trình các bước xây dựng giải pháp tự động hóa với Hugging Face

Hình 7 là Quy trình các bước xây dựng giải pháp tự động hóa với Hugging Face được mô tả qua các bước như sau:

### **Bước 1: Chọn mô hình phù hợp**

- Xác định nhu cầu:
  - Chatbot?
  - Tóm tắt văn bản?
  - Trả lời câu hỏi từ tài liệu riêng (RAG)?
- Chọn mô hình theo bài toán:
  - text-generation: sinh câu trả lời mới.
  - text2text-generation: chuyển đổi dạng đầu ra.
  - image-to-text (nếu đa phương thức).

### **Bước 2: Setup môi trường**

- Cài đặt:

```
pip install transformers accelerate torch
```

- Nếu cần giao diện web:

```
pip install gradio
```

### **Bước 3: Load mô hình từ Hugging Face**

- Ví dụ tải Llama3-7B-Instruct:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

model_name = "meta-llama/Llama-3-7b-chat-hf"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name,
torch_dtype=torch.float16, device_map="auto")

inputs = tokenizer("Xin chào! Bạn khỏe không?",
return_tensors="pt").to(model.device)
outputs = model.generate(**inputs, max_new_tokens=100)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

#### Bước 4: Kết nối vào giải pháp

- Đóng gói thành API nội bộ (FastAPI, Flask).
- Tích hợp vào UI (Gradio, Streamlit).
- Thêm cơ chế tự động:
  - Gửi email tự động từ AI.
  - Tạo báo cáo PDF AI-generated.
  - Kết nối API bên ngoài để tự động hóa workflow.

### 3.4.Mô hình xây dựng giải pháp tự động hóa với Hugging Face:

#### 3.4.1.Load và chạy thử một mô hình chat open-source

Ví dụ dùng mô hình mistralai/Mistral-7B-Instruct-v0.1:

```
from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline

model_id = "mistralai/Mistral-7B-Instruct-v0.1"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id)

chat = pipeline("text-generation", model=model, tokenizer=tokenizer)

prompt = "Hãy kể một câu chuyện vui về trí tuệ nhân tạo."
output = chat(prompt, max_new_tokens=100, do_sample=True)[0]["generated_text"]
print(output)
```

Kết quả sau khi chạy ta được:

```
Fetching 2 files: 100%|██████████| 2/2 [02:41<00:00, 80.54s/it]
Loading checkpoint shards: 100%|██████████| 2/2 [01:35<00:00, 47.81s/it]
Device set to use cpu
Setting `pad_token_id` to `eos_token_id`:2 for open-end generation.
Hãy kể một câu chuyện vui về trí tuệ nhân tạo.

Once upon a time, in a small town nestled in a picturesque valley between two mountain ranges, there lived a young boy named Tuan
One day, a group of travelers passed through the town, seeking refuge from a terrible storm that had been raging all day
```

### 3.4.2. Tạo một API nội bộ nhỏ

Dưới đây là code tạo ảnh AI về: “Một lớp học tương lai với đầy đủ học sinh học mã hóa AI theo phong cách siêu thực của Salvador Dali”:

```
import torch
from diffusers import FluxPipeline

pipe = FluxPipeline.from_pretrained("black-forest-labs/FLUX.1-schnell",
torch_dtype=torch.bfloat16).to("cuda")
generator = torch.Generator(device="cuda").manual_seed(0)
prompt = "A futuristic class full of students learning AI coding in the surreal style of Salvador Dali"
Generate the image using the GPU
image = pipe(
 prompt,
 guidance_scale=0.0,
 num_inference_steps=4,
 max_sequence_length=256,
 generator=generator
).images[0]

image.save("surreal.png")
```



```
from IPython.display import display
from PIL import Image

Display the image inline
display(image)
```

Kết quả sau khi chạy code ta có ảnh tạo từ AI như sau:



*Hình ảnh 8: Một lớp học tương lai với đầy đủ học sinh học mã hóa AI*

### 3.4.3. Tổng hợp giọng nói (Text-to-Speech)

Tổng hợp giọng nói (Text-to-Speech) bằng mô hình SpeechT5 của Microsoft từ một chuỗi văn bản, đồng thời dùng embedding của một người nói cụ thể để mô phỏng giọng nói của người đó. Đây là các bước chi tiết của đoạn code:

```
from transformers import pipeline
from datasets import load_dataset
import soundfile as sf
import torch

synthesiser = pipeline("text-to-speech", "microsoft/speecht5_tts", device='cuda')

embeddings_dataset = load_dataset("Matthijs/cmu-arctic-xvectors", split="validation")
speaker_embedding =
torch.tensor(embeddings_dataset[7306]["xvector"]).unsqueeze(0)

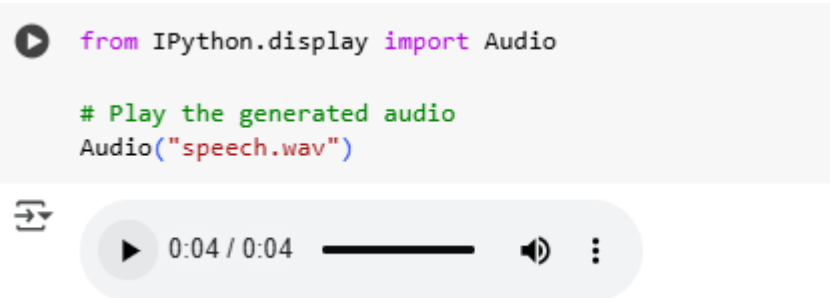
speech = synthesiser("Hi to an artificial intelligence engineer on the way to mastery!",
forward_params={"speaker_embeddings": speaker_embedding})

sf.write("speech.wav", speech["audio"], samplerate=speech["sampling_rate"])

from IPython.display import Audio

Play the generated audio
Audio("speech.wav")
```

Chuyển văn bản thành giọng nói bằng mô hình **SpeechT5** với giọng mô phỏng theo embedding của một người nói trong tập CMU Arctic, và lưu file đầu ra là speech.wav. Kết quả sau khi chạy ta được kết quả như sau:



#### 3.4.4. Tạo hình ảnh từ văn bản (Text-to-Image generation)

Tạo hình ảnh từ văn bản (Text-to-Image generation) bằng mô hình Stable Diffusion Turbo (sd-turbo) từ thư viện diffusers của Hugging Face. Đây là mô hình được tối ưu để tạo ảnh nhanh.

```
from diffusers import AutoPipelineForText2Image
import torch

pipe = AutoPipelineForText2Image.from_pretrained("stabilityai/sd-turbo",
torch_dtype=torch.float16, variant="fp16")
pipe.to("cuda")

prompt = "A futuristic class full of students learning AI coding in a surreal style"
image = pipe(prompt=prompt, num_inference_steps=1, guidance_scale=0.0).images[0]

from IPython.display import display
from PIL import Image
Display the image inline
display(image)
```

Kết quả sau khi tạo một lớp học tương lai với đầy đủ học sinh học mã hóa AI theo phong cách siêu thực với Hugging face:



*Hình ảnh 9: Code tạo hình ảnh từ văn bản (Text-to-Image generation)*

### 3.4.5. Thực hiện quy trình tự động chuyển đổi nội dung một buổi họp từ file âm thanh sang biên bản họp chi tiết (meeting minutes)

Bằng cách sử dụng các công nghệ AI như Whisper (chuyển giọng nói thành văn bản) và LLM (tổng hợp biên bản họp từ transcript). Dưới đây là giải thích chi tiết theo từng bước:

#### **Bước 1: Chuyển giọng nói thành văn bản bằng Whisper**

```
from transformers import pipeline
```

```
Tạo pipeline với mô hình Whisper (nó tự tải từ Huggingface, giống OpenAI nhưng chạy local hoặc cloud của bạn)
```

```
transcriber = pipeline("automatic-speech-recognition", model="openai/whisper-tiny")
```

Tạo pipeline chuyển giọng nói thành văn bản (ASR) bằng mô hình **Whisper Tiny** của OpenAI (chạy cục bộ):

```
Đường dẫn file âm thanh
audio_filename =
"/content/drive/MyDrive/llm_engineering/llm_engineering/denver_extract.mp3"
result = transcriber(audio_filename, return_timestamps=True)
print("Transcript:", result['text'])
```

- Đọc file âm thanh .mp3 từ Google Drive.
- Tạo transcript có kèm timestamp từ file âm thanh.

## Bước 2: Tạo prompt cho LLM để viết biên bản họp

```
system_message = "You are an assistant that produces minutes of meetings from
transcripts, with summary, key discussion points, takeaways and action items with
owners, in markdown."
user_prompt = f"Below is an extract transcript of a Denver council meeting. Please
write minutes in markdown, including a summary with attendees, location and date;
discussion points; takeaways; and action items with owners.\n{transcription}"

messages = [
 {"role": "system", "content": system_message},
 {"role": "user", "content": user_prompt}
]
```

Mô tả đoạn code tạo prompt cho LLM để viết biên bản cuộc họp:

- system\_message: định nghĩa vai trò của LLM là “thư ký biên bản họp”.
- user\_prompt: tạo nội dung yêu cầu: tóm tắt transcript thành biên bản họp, bao gồm thông tin ngày, người tham dự, nội dung thảo luận, kết luận và công việc hành động.
- Gói system và user message lại thành danh sách đầu vào cho mô hình hội thoại kiểu chat (giống như ChatGPT).

## Bước 3: Cấu hình mô hình ngôn ngữ lớn (LLM) để sinh văn bản

```
quant_config = BitsAndBytesConfig(
 load_in_4bit=True,
 bnb_4bit_use_double_quant=True,
 bnb_4bit_compute_dtype=torch.bfloat16,
 bnb_4bit_quant_type="nf4")
```

Dùng **4-bit quantization** để giảm tải bộ nhớ và tăng hiệu suất khi chạy mô hình lớn như LLaMA trên GPU yếu.

```
tokenizer = AutoTokenizer.from_pretrained(LLAMA)
tokenizer.pad_token = tokenizer.eos_token
inputs = tokenizer.apply_chat_template(messages, return_tensors="pt").to("cuda")
streamer = TextStreamer(tokenizer)
model = AutoModelForCausalLM.from_pretrained(LLAMA, device_map="auto",
quantization_config=quant_config)
outputs = model.generate(inputs, max_new_tokens=2000, streamer=streamer)
```

Chuẩn bị tokenizer và áp dụng template dạng hội thoại để chuyển messages thành đầu vào cho LLM:

```
streamer = TextStreamer(tokenizer)
model = AutoModelForCausalLM.from_pretrained(LLAMA,
device_map="auto",quantization_config=quant_config)
outputs = model.generate(inputs, max_new_tokens=2000, streamer=streamer)
```

- Tải mô hình LLaMA hoặc LLM tương tự đã fine-tune để sinh ra **meeting minutes**.
- Sinh văn bản kết quả tối đa 2000 tokens.



Kết quả đoạn code demo:

## Minutes of the Denver City Council Meeting

Date: October 9, 2017 Location: Denver City Council Chambers Attendees:

- Councilman Lopez
- Councilwoman Clark
- Councilman Gilmore
- Councilwoman Flynn
- Councilman Cashman
- Councilman New
- Councilwoman Ortega
- Councilman Cessman
- Councilwoman Maritaga
- Councilman Minkinich

**Summary:** The Denver City Council meeting was held on October 9, 2017, to discuss and approve proclamation 1127, series of 2017, observing the second annual Indigenous Peoples Day in the City and County of Denver.

### Key Discussion Points:

- Councilman Lopez read proclamation 1127, series of 2017, which recognizes the contributions of Indigenous people to the city's history, past, present, and future.
- Councilman Lopez emphasized the importance of pride in one's own culture without requiring contempt or disrespect of another.
- Councilwoman Maritaga shared her experience with the Commission on Indian Affairs and the importance of preserving Native American culture and language.
- Councilman Minkinich discussed the intersection of cultural preservation and environmental protection.

### Takeaways:

- The Denver City Council recognizes the importance of Indigenous Peoples Day and the contributions of Indigenous people to the city's history and culture.
- The council emphasizes the need to address critical issues affecting Indigenous communities, such as poverty, lack of access to services, and housing.
- The proclamation highlights the significance of cultural preservation and environmental protection.

### Action Items:

- Adopt proclamation 1127, series of 2017, observing the second annual Indigenous Peoples Day in the City and County of Denver.
- Support the proclamation and its aims to promote education and awareness about Indigenous people's contributions to the city's history and culture.
- Address critical issues affecting Indigenous communities, such as poverty, lack of access to services, and housing.

### Owners:

- Councilman Lopez: Sponsor of proclamation 1127, series of 2017.
- Councilwoman Maritaga: Co-sponsor of proclamation 1127, series of 2017.
- Councilman Minkinich: Supporter of proclamation 1127, series of 2017.

*Hình 10: Chuyển file âm thanh sang biên bản họp chi tiết (meeting minutes)*

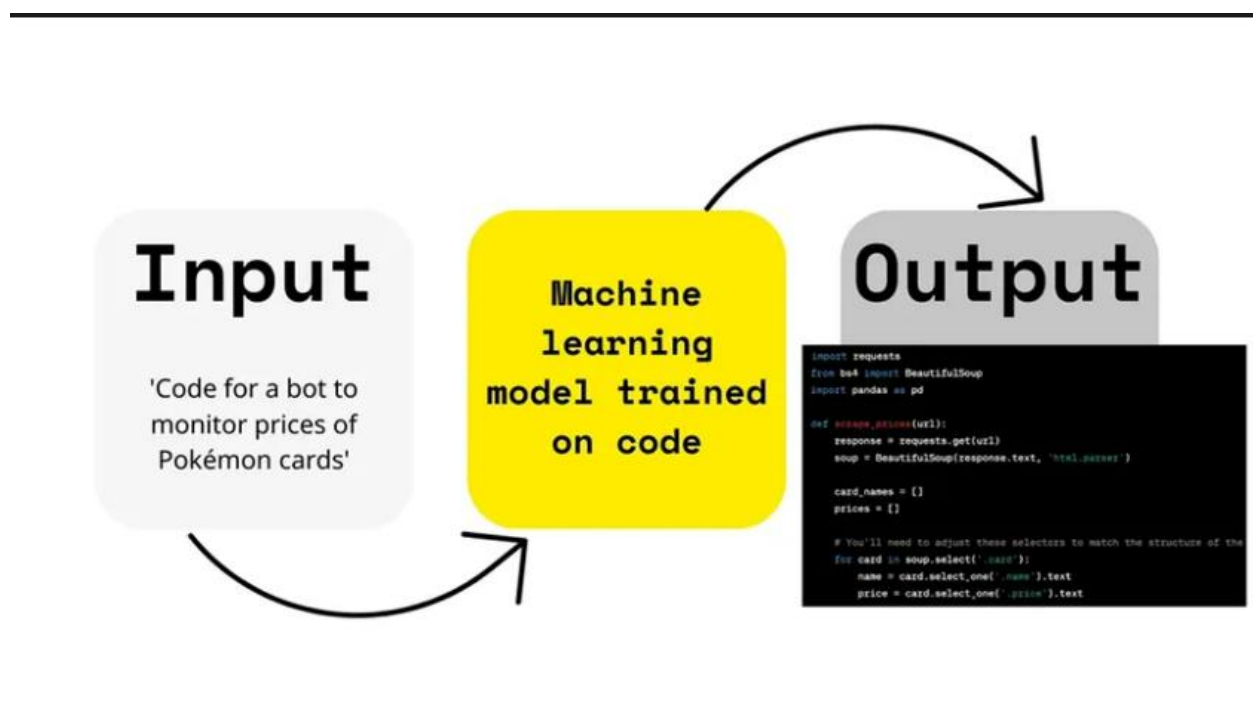
## Chương 4: Đánh giá Mô hình cho Sinh mã (Code Generation) và Nhiệm vụ Kinh doanh (Business Tasks)

### 4.1. Tầm quan trọng của việc đánh giá LLM

- Không phải mọi LLM đều mạnh trong tất cả các tác vụ.
- Một số mô hình tốt cho sáng tạo nội dung, một số khác tốt cho sinh mã hoặc phân tích dữ liệu.
- Cần chọn đúng mô hình để tối ưu hóa:
  - Độ chính xác (Accuracy)
  - Chi phí vận hành (Cost)
  - Tốc độ phản hồi (Latency)

### 4.2. Các nhóm nhiệm vụ cần đánh giá

#### ❖ Nhiệm vụ 1: Sinh mã lập trình (Code Generation):



Hình 11: Code Generation mô hình học máy



Hình 11 ở trên mô tả một mô hình học máy như sau:

- Viết hàm, thuật toán, tạo API.
- Ví dụ: viết function xử lý chuỗi, sinh unit test.
- ❖ **Nhiệm vụ 2: Tác vụ kinh doanh (Business Tasks):**



*Hình 12: Business task trong LLM*

Hình 12 với nhiệm vụ tác kinh doanh:

- Tóm tắt văn bản, phân loại email.
- Phân tích dữ liệu khách hàng.
- Viết báo cáo nội bộ.

#### **4.3. Các mô hình tham gia “Showdown” (So tài)**

- **GPT-4 Turbo** (OpenAI): mạnh toàn diện, đặc biệt trong code và reasoning.
- **Claude 3 Opus/Sonnet** (Anthropic): tốt trong business communication, viết báo cáo.
- **Gemini 1.5 Pro** (Google): mạnh về multi-modal và trả lời theo ngữ cảnh lớn.
- **Mistral / Mixtral** (open-source): nhanh, chi phí thấp, hiệu quả tốt.
- **CodeQwen / StarCoder2 / DeepSeekCoder**: chuyên sinh mã nguồn.
- **LLaMA 3 Instruct** (Meta): open-source LLM thế hệ mới, có version hỗ trợ code.

## 4.4.Cách tiến hành đánh giá mô hình (LLM Evaluation)

### a) Code Generation Benchmark

- Giao bài toán nhỏ (ví dụ: viết một sorting function, API CRUD).
- Các tiêu chí:
  - Tính đúng đắn (Correctness)
  - Tính tối ưu (Optimization)
  - Khả năng đọc code (Readability)

### b) Business Task Benchmark

- Giao các nhiệm vụ thực tế:
  - Tóm tắt tài liệu dài (meeting notes → executive summary).
  - Viết email bán hàng hoặc chăm sóc khách hàng.
  - Phân tích bảng dữ liệu đơn giản.
- Các tiêu chí:
  - Độ chính xác
  - Khả năng nắm bắt ngữ cảnh
  - Tính chuyên nghiệp ngôn ngữ

### c) Đo chi phí và tốc độ

- Thời gian phản hồi (Response Time).
- Chi phí inferencing (\$/1K tokens).

## 4.5.Code demo so sánh 2 mô hình GEMINI và OLLAMA Mistral về Business Tasks và Code Generation

```
=== FUNCTION: GEMINI ===
def ask_gemini(prompt):
 try:
 response = gemini_model.generate_content(prompt)
 return response.text
 except Exception as e:
 return f"[Gemini Error] {str(e)}"

=== FUNCTION: OLLAMA ===
def ask_ollama(prompt):
 try:
 response = requests.post("http://localhost:11434/api/generate", json={
 "model": OLLAMA_MODEL,
 "prompt": prompt,
 "stream": False
 })
 return response.json().get("response", "[No Response]")
 except Exception as e:
 return f"[Ollama Error] {str(e)}"

=== FUNCTION: SO SÁNH 2 MÔ HÌNH ===
def compare_models(task_type, user_input):
 if task_type == "Code Generation":
 prompt = f"Viết code cho yêu cầu sau bằng Python:\n{user_input}"
 else:
 prompt = f"Xử lý tác vụ kinh doanh sau bằng văn phong chuyên nghiệp:\n{user_input}"
```

```

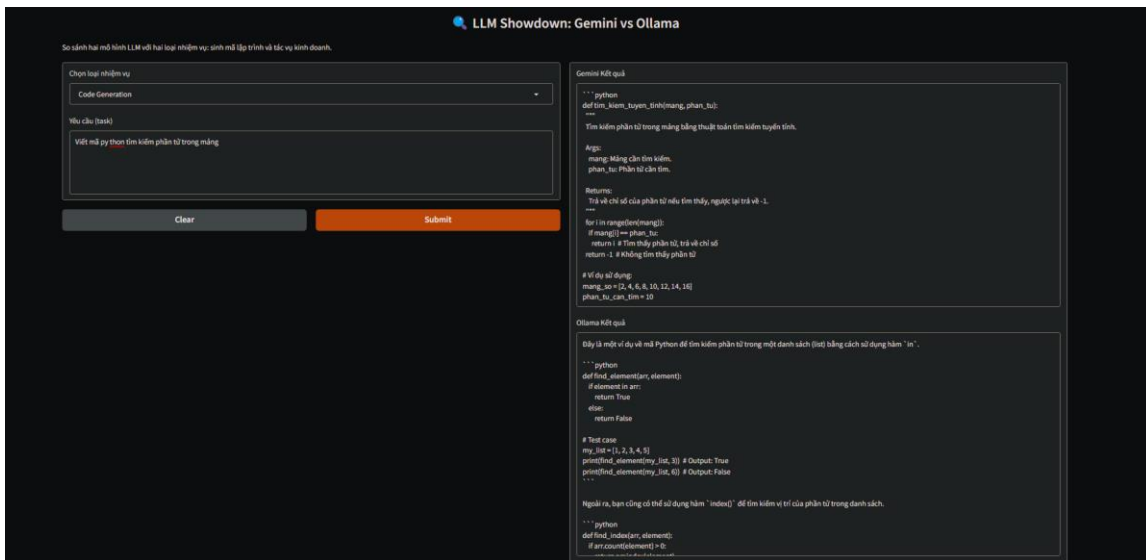
gemini_result = ask_gemini(prompt)
ollama_result = ask_ollama(prompt)

return gemini_result, ollama_result

```

Giải thích code: so sánh 2 mô hình ngôn ngữ AI về Business Task và Code Geranation mỗi một mô hình đều có ưu điểm thế mạnh riêng

#### 4.5.1. Kết quả khi so sánh Code Generation:

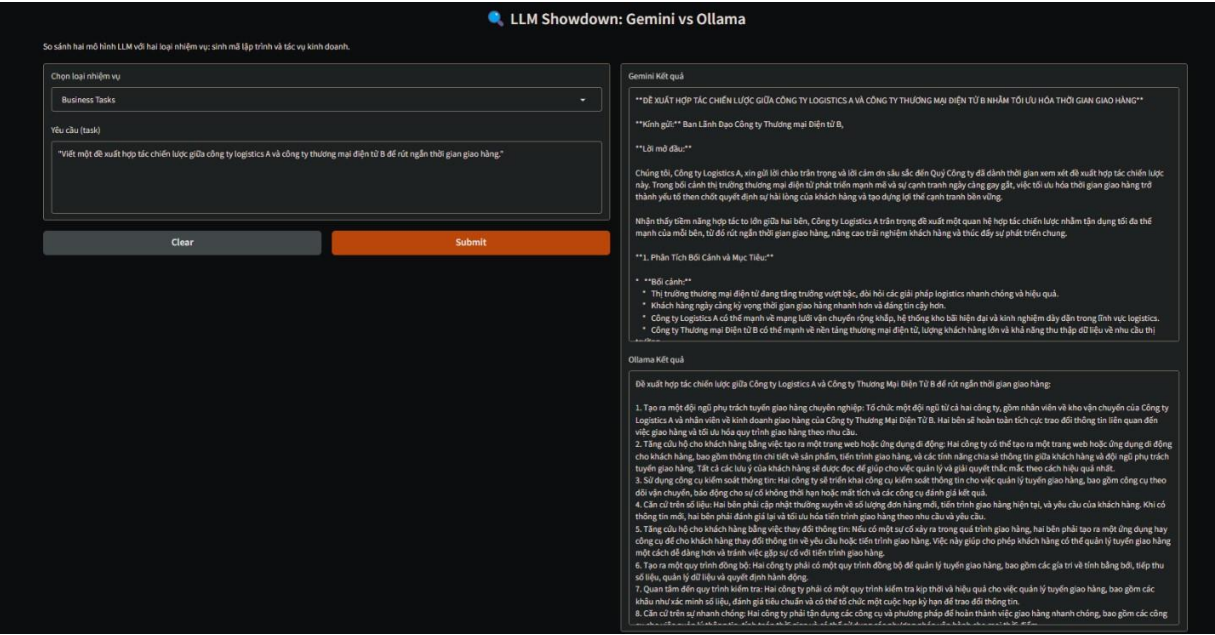


**Germini:** Tốt hơn so với Ollama

| Tiêu chí                         | Gemini 1.5 Pro                                              | Ollama (Code LLaMA / Mistral )                                                         |
|----------------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>Độ chính xác (Accuracy)</b>   | Rất cao, Hiểu yêu cầu tốt, sinh mã đầy đủ, kèm giải thích.  | Khá cao, nhưng phụ thuộc mô hình (Mistral < Code LLaMA). Thiếu khả năng diễn giải sâu. |
| <b>Chi phí vận hành (Cost)</b>   | Cao nếu gọi qua API (tính theo tokens, trả phí theo usage). | Thấp / miễn phí nếu chạy local. Tốn tài nguyên máy tính.                               |
| <b>Tốc độ phản hồi (Latency)</b> | Chậm hơn do phụ thuộc mạng + kích thước mô hình.            | Nhanh hơn (chạy local), độ trễ thấp nếu máy đủ mạnh.                                   |

Bảng 2: So sánh tiêu chí giữa 2 mô hình ngôn ngữ về Code Generation

4.5.2.Kết quả khi so sánh Business Tasks



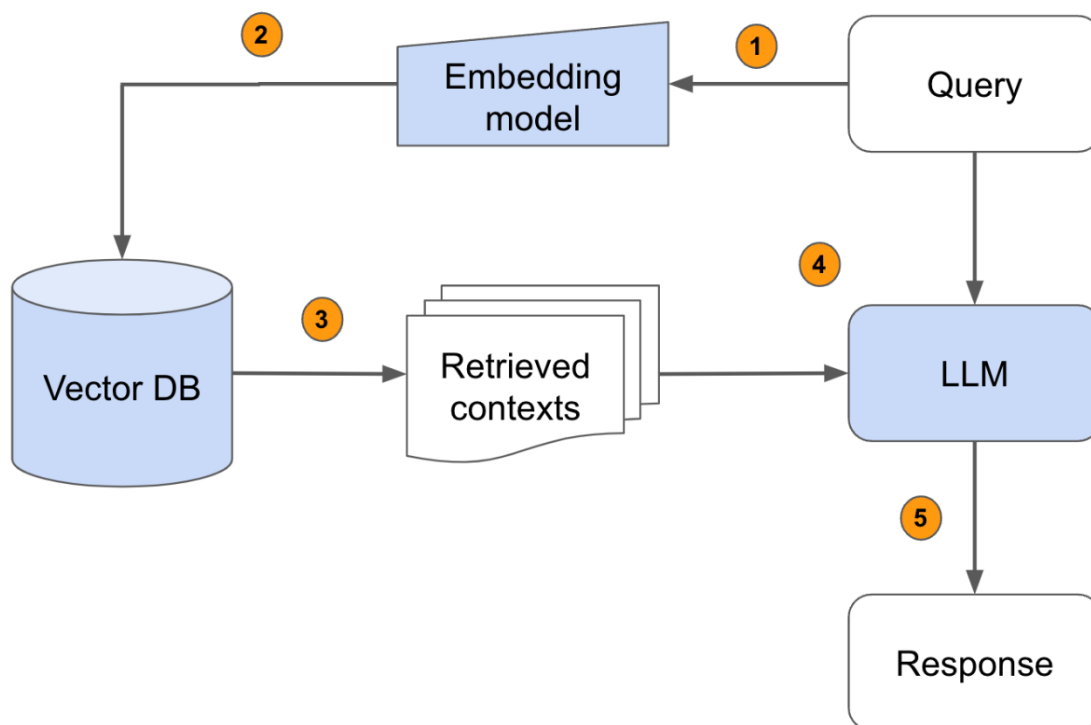
Ollama tốt hơn Gemini về Business Tasks

| Tiêu chí         | Gemini 1.5 Pro                                   | Ollama (Mistral / Phi-2)                            |
|------------------|--------------------------------------------------|-----------------------------------------------------|
| Độ chính xác     | Rất cao tốt cho các task phức tạp, phân tích sâu | Tốt với task đơn giản; kém hơn trong logic phức tạp |
| Chi phí vận hành | Cao cần API trả phí theo usage hoặc subscription | Thấp chạy local, không cần gọi API                  |
| Tốc độ phản hồi  | Tốt (nếu mạng ổn định); phụ thuộc server & load  | Rất nhanh nếu máy đủ mạnh vì chạy hoàn toàn cục bộ  |

Bảng 3: So sánh tiêu chí giữa 2 mô hình ngôn ngữ về Business Tasks

## Chương 5: Xây dựng Giải pháp Nâng cao với Vector Embeddings & LangChain

### 5.1.RAG là gì? (Retrieval-Augmented Generation)



Hình 13: Hệ thống RAG

Hình 13 mô tả về hệ thống RAG như sau:

**Bước 1: Query → Embedding model**

Người dùng nhập một truy vấn (query). Truy vấn này được đưa vào mô hình embedding để chuyển thành vector số hóa, đại diện cho ý nghĩa của truy vấn.

**Bước 2: Embedding → Vector DB**

Vector biểu diễn truy vấn được gửi đến cơ sở dữ liệu vector (vector database). Vector DB này chứa các vector embedding của các tài liệu đã được xử lý và lưu trữ từ trước.

**Bước 3: Vector DB → Retrieved contexts**

Cơ sở dữ liệu vector tìm kiếm các văn bản có embedding gần giống với truy vấn. Các văn bản này được truy xuất và được gọi là "retrieved contexts" (ngữ cảnh được truy xuất).

**Bước 4: Retrieved contexts + Query → LLM**

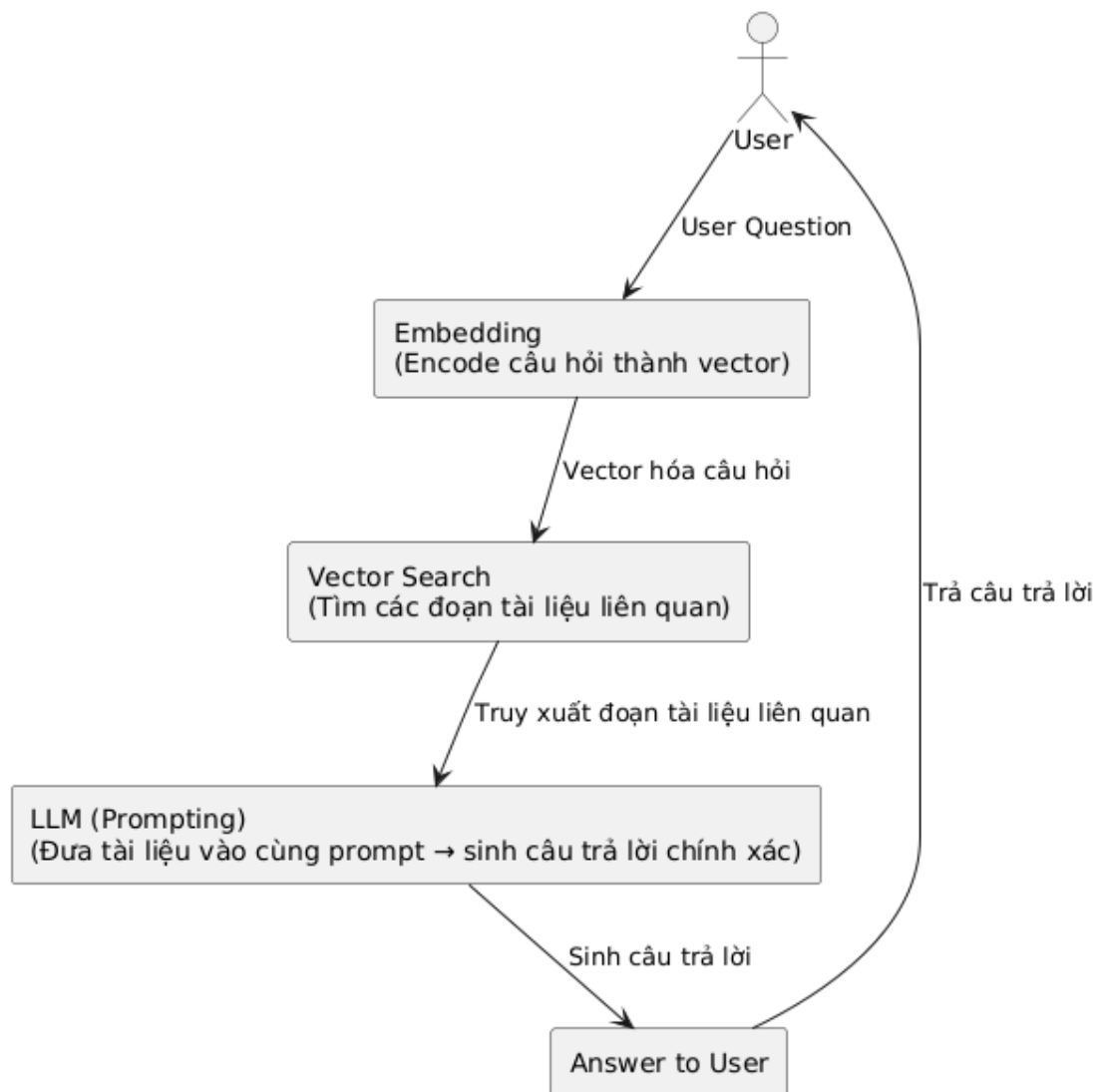
Các ngữ cảnh được truy xuất cùng với truy vấn ban đầu được đưa vào mô hình ngôn ngữ lớn (LLM). LLM sẽ sử dụng thông tin từ cả hai nguồn này để hiểu rõ yêu cầu và tạo ra phản hồi phù hợp.

**Bước 5: LLM → Response**

LLM sinh ra phản hồi dựa trên truy vấn và các ngữ cảnh liên quan, sau đó phản hồi này được trả lại cho người dùng.

## 5.2.Kiến trúc cơ bản của một hệ thống RAG

### Kiến trúc cơ bản của hệ thống RAG (Retrieval-Augmented Generation)





### 5.3.Các thành phần quan trọng trong RAG

| Thành phần              | Công cụ điển hình                              | Mô tả                              |
|-------------------------|------------------------------------------------|------------------------------------|
| <b>Embeddings model</b> | OpenAI Ada-002, HuggingFace models, Instructor | Biến văn bản thành vector số học   |
| <b>Vector Database</b>  | FAISS, ChromaDB, Pinecone, Weaviate            | Lưu trữ và tìm kiếm theo vector    |
| <b>LLM backend</b>      | GPT-4, Claude, Mistral, Llama 3, DeepSeekCoder | Sinh ra câu trả lời                |
| <b>Framework hỗ trợ</b> | LangChain, LlamaIndex                          | Quản lý pipeline, chaining dễ dàng |

*Bảng 4: Thành phần công cụ điển hình và mô tả công cụ trong hệ thống RAG*

### 5.4.Tóm tắt LangChain (Framework cho RAG)

- **LangChain** là một thư viện Python mạnh mẽ để:
  - Tạo chuỗi tương tác giữa LLM ↔ dữ liệu ↔ user input.
  - Xây dựng dễ dàng các workflow phức tạp (ví dụ: memory, agents, multi-step RAG).

#### **Một số module chính của LangChain:**

- Document Loaders: đọc PDF, Docx, Notion, website...
- Text Splitters: chia nhỏ văn bản để indexing.
- Embeddings: encode dữ liệu thành vector.
- VectorStores: lưu vector (Chroma, FAISS,...).
- Retrievers: tìm kiếm vector liên quan.
- Chains: quản lý cách LLM lấy dữ liệu → trả lời.

### **5.5.Những lưu ý khi xây dựng RAG chất lượng cao**

- Text Splitting tối ưu: Chia văn bản hợp lý để tránh mất ngữ cảnh.
- Embeddings chất lượng cao: Chọn model encode tốt (ví dụ: Instructor-XL, OpenAI Ada-002).
- Prompt engineering: Thiết kế prompt để LLM hiểu cách sử dụng dữ liệu trả về.
- Multi-hop Retrieval: Nếu dữ liệu quá lớn, có thể cần truy xuất nhiều lần.
- Caching & Speed optimization: Với hệ thống lớn, cần cache query vector và result.

## 5.6. Demo Giải pháp Nâng cao với Vector Embeddings & LangChain

### Code demo:

```
def cosine_similarity(vec1, vec2):
 """Tính cosine similarity giữa hai vector"""
 dot_product = np.dot(vec1, vec2)
 norm_a = np.linalg.norm(vec1)
 norm_b = np.linalg.norm(vec2)
 # Tránh chia cho 0
 if norm_a == 0 or norm_b == 0:
 return 0
 return dot_product / (norm_a * norm_b)
Hàm chính cho pipeline của demo
def ollama_vector_search(question):
 try:
 # Bước 1: Gửi yêu cầu POST tới Ollama API để sinh embedding
 response = requests.post("http://localhost:11434/api/generate", json={
 "model": OLLAMA_MODEL,
 "prompt": question,
 "stream": False })
 # Kiểm tra phản hồi từ Ollama
 if response.status_code == 200:
 embedding_response = response.json()
 query_embedding = embedding_response.get("embedding", [])
 else:
 return f"Error: {response.status_code} - {response.text}"
 if not query_embedding:
 return "Error: Failed to generate embedding for the question."
 except Exception as e:
 return f"Error generating embedding: {str(e)}"
```

```

Bước 2: Tìm kiếm trong Vector DB (Giả lập)
Các tài liệu mẫu sẽ được tạo ngẫu nhiên (thực tế, bạn sẽ sử dụng vector database
thực tế)

document_embeddings = [
 {"text": "Tài liệu về trí tuệ nhân tạo và các ứng dụng.",
 "embedding": np.random.rand(len(query_embedding))},
 {"text": "Hướng dẫn sử dụng Ollama để chạy mô hình ngôn ngữ cục bộ.",
 "embedding": np.random.rand(len(query_embedding))},
 {"text": "Vector search cho phép tìm kiếm ngữ nghĩa dựa trên sự tương đồng.",
 "embedding": np.random.rand(len(query_embedding))}
]

best_doc = None
best_similarity = -1 # Bắt đầu từ -1 vì cosine similarity nằm trong khoảng từ -1 đến
1

Tính toán độ tương đồng và tìm tài liệu phù hợp nhất
for doc in document_embeddings:
 similarity = cosine_similarity(query_embedding, doc["embedding"])
 if similarity > best_similarity:
 best_doc = doc["text"]
 best_similarity = similarity

if not best_doc:
 context = "Không tìm thấy tài liệu phù hợp."
else:
 context = f"Tài liệu phù hợp (độ tương đồng: {best_similarity:.4f}): {best_doc}"

```

```

Bước 3: Sử dụng Ollama để sinh câu trả lời với ngữ cảnh
try:
 response = requests.post("http://localhost:11434/api/generate", json={
 "model": OLLAMA_MODEL,
 "prompt": f"Câu hỏi: {question}\nNgữ cảnh: {context}",
 "stream": False
 })

 if response.status_code == 200:
 answer_response = response.json()
 answer = answer_response.get("response", "[No Answer]")
 else:
 return f"Error: {response.status_code} - {response.text}"

except Exception as e:
 return f"Error generating answer: {str(e)}"

return f"""Kết quả tìm kiếm:
{context}

Câu trả lời:
{answer}"""

Giao diện Gradio
interface = gr.Interface(
 fn=ollama_vector_search,
 inputs=gr.Textbox(placeholder="Nhập câu hỏi của bạn ở đây..."),
 outputs=gr.Textbox(label="Kết quả"),
 title="Demo Ollama với Vector Search",

```

```

description="Nhập câu hỏi và nhận câu trả lời sử dụng vector search và Ollama.",
examples=[
 ["Trí tuệ nhân tạo là gì?"],
 ["Làm thế nào để sử dụng Ollama?"],
 ["Vector search hoạt động như thế nào?"]
]
)

if __name__ == "__main__":
 interface.launch(share=True) # share=True để chia sẻ URL công khai tạm thời

```

**Giải thích code:** Chuyển văn bản thành vector, sau đó nhập câu hỏi hệ thống câu hỏi được nhập vào chuyển hóa thành vector và truy vấn tới vector database giả lập từ đó ngữ cảnh mới được truy xuất và thành câu hỏi

### Kết quả tìm kiếm:

Input : Trí tuệ nhân tạo là gì

### Câu hỏi được gửi đi về ollama:

Tài liệu phù hợp (độ tương đồng: 0.7360): “Tài liệu về trí tuệ nhân tạo và các ứng dụng”.

Từ đó in ra kết quả output được như hình ảnh dưới đây.

**Demo Ollama với Vector Search**

Nhập câu hỏi và nhận câu trả lời sử dụng vector search và Ollama.

question

Trí tuệ nhân tạo là gì?

Clear Submit

**Kết quả**

Kết quả tìm kiếm:  
Tài liệu phù hợp (độ tương đồng: 0.7360): Tài liệu về trí tuệ nhân tạo và các ứng dụng.

Câu trả lời:  
Trí tuệ nhân tạo là một kỹ năng, tổng hợp kiến thức và kỹ năng để giải quyết các vấn đề, tìm ra các hình thức để trích xuất sự khác biệt và phân biệt những bộ phận trong các quyết định. Trí tuệ nhân tạo có thể được áp dụng trong rất nhiều lĩnh vực, trong đó mà trẻ em sử dụng để phát triển kỹ năng động lý và quản lý tìm kiếm. Nó cũng có thể giúp người trẻ nên hơn khi, tự xây dựng mọi mục tiêu và sử dụng những góc động lý của mình để làm việc.

Trong bài đầu, có rất nhiều phần mềm và công cụ trí tuệ nhân tạo được sử dụng cho việc giảng dạy và hỗ trợ người trẻ em sử dụng trí tuệ nhân tạo. Vì cơ bản, các công cụ này giúp trẻ em phát triển kỹ năng động lý và quản lý tìm kiếm, bao gồm tìm kiếm, xây dựng và xử lý các mục tiêu. Trí tuệ nhân tạo có thể giúp trẻ em phát triển kỹ năng học tập, khuyến khích và bản thân, quản lý các quyết định tương tác và giải quyết các vấn đề.

Trong những bài trí tuệ đầu tiên, trẻ em sử dụng cách để trích xuất các góc động lý của mình để tìm kiếm. Các công cụ trí tuệ nhân tạo phù hợp cho việc giảng dạy và hỗ trợ người trẻ em sử dụng trí tuệ nhân tạo có thể giúp trẻ em phát triển kỹ năng động lý và quản lý tìm kiếm, bao gồm tìm kiếm, xây dựng và xử lý các mục tiêu. Nó cũng có thể giúp trẻ em phát triển kỹ năng học tập, khuyến khích và bản thân, quản lý các quyết định tương tác và giải quyết các vấn đề.

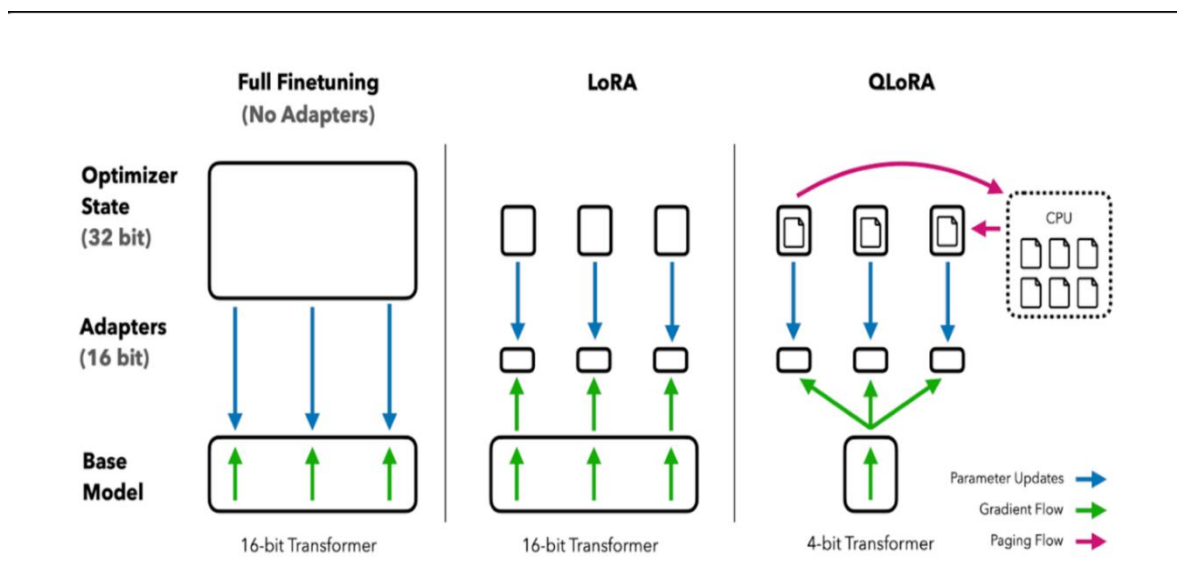
Flag

Examples

Trí tuệ nhân tạo là gì? Làm thế nào để sử dụng Ollama? Vector search hoạt động như thế nào?

## Chương 6: Tinh chỉnh các mô hình LLM tiên tiến bằng kỹ thuật LoRA và QLoRA

### 6.1.Vì sao cần Fine-tuning LLM?



Hình ảnh 14: Mô Hình LoRA và QLoRA

Hình 14 mô tả về các mô hình tinh chỉnh như sau:

- Mô hình gốc (base models) như GPT-4, Llama-3, Mistral... thường rất mạnh nhưng:
  - Không tối ưu cho nhiệm vụ chuyên biệt (ví dụ: luật, y khoa, tài chính, kỹ thuật).
  - Ngôn ngữ, phong cách trả lời không đúng yêu cầu doanh nghiệp.
- Fine-tuning = tái huấn luyện mô hình trên tập dữ liệu mới → cho kết quả chính xác hơn.

| Phương pháp     | Có train thêm không? | Train phần nào?         | Tiết kiệm tài nguyên? | Dùng cho mô hình lớn? |
|-----------------|----------------------|-------------------------|-----------------------|-----------------------|
| Full Finetuning | Có                   | Toàn bộ mô hình         | Không                 | Tốn kém               |
| LoRA            | Có                   | Chỉ adapter nhỏ         | Có                    | Có thể                |
| QLoRA           | Có                   | Adapter + mô hình 4-bit | Rất tiết kiệm         | Tốt nhất              |

Bảng 5: So sánh các mô hình Finetuning

## 6.2.LoRA và QLoRA là gì?

| Kỹ thuật                          | Giải thích đơn giản                                                                                               | Ưu điểm                                                                                                                                                                |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>LoRA (Low-Rank Adaptation)</b> | -Phương pháp tinh chỉnh nhẹ, chỉ cập nhật một lượng nhỏ tham số. thông qua các lớp adapter được chèn vào mô hình. | <ul style="list-style-type: none"> <li>- Nhanh hơn nhiều lần</li> <li>- Cần ít GPU memory hơn</li> <li>- Dễ áp dụng</li> </ul>                                         |
| <b>QLoRA (Quantized LoRA)</b>     | -Kết hợp Quantization(Lượng tử hóa mô hình) (giảm độ chính xác model xuống 4-bit).                                | <ul style="list-style-type: none"> <li>- Tiết kiệm VRAM cực lớn</li> <li>- Fine-tune Llama-2-7B chỉ với 24GB VRAM</li> <li>- Phù hợp cho laptop/PC mạnh vừa</li> </ul> |

*Bảng 6: So sánh kỹ thuật các mô hình Finetuning*

### 6.3.Kiến trúc Fine-tuning với LoRA / QLoRA



Hình 15: Kiến trúc hệ thống LoRA và QLoRA



Giải thích hình 15:

| Thành phần                | Vai trò                                                                                  |
|---------------------------|------------------------------------------------------------------------------------------|
| <b>Base model</b>         | Là mô hình nền (LLM) như LLaMA, GPT, v.v. – không bị cập nhật trực tiếp                  |
| <b>LoRA adapters</b>      | Các lớp nhỏ được thêm vào giữa các lớp transformer – chỉ những lớp này được huấn luyện   |
| <b>Optimizer state</b>    | Trạng thái nội bộ của thuật toán tối ưu (Adam, SGD, v.v.), dùng để cập nhật LoRA adapter |
| <b>QLoRA: Paging flow</b> | Với QLoRA, optimizer state được lưu trên CPU để giảm VRAM sử dụng                        |
| <b>Precision</b>          | LoRA dùng base model 16-bit, QLoRA dùng mô hình 4-bit (nhẹ hơn)                          |

#### 6.4.Các thành phần trong quy trình Fine-tuning

**Mô hình nền (Base Model):** thường là mô hình đã được huấn luyện trước (pretrained model).

- Adapter: lớp học bổ sung được thêm vào mô hình để tinh chỉnh mà không cần cập nhật toàn bộ trọng số.
- Optimizer: thuật toán tối ưu như Adam hoặc AdamW giúp cập nhật tham số.
- Loss Function: hàm mất mát để tính sai số giữa đầu ra mô hình và nhãn đúng.
- Dataloader: công cụ tải dữ liệu vào mô hình trong quá trình huấn luyện.
- Training loop: vòng lặp huấn luyện mô hình qua nhiều epoch (vòng lặp qua dữ liệu huấn luyện).

## 6.5. Các mẹo Fine-tuning thực tế

- Rất chú ý Dataset chất lượng: 80% thành công của Fine-tuning đến từ chất lượng dữ liệu, không phải model.
- Chỉ fine-tune những module cần thiết: Tiết kiệm GPU, nhanh convergence.
- Sử dụng LoRA/QLoRA để tiết kiệm bộ nhớ và tài nguyên tính toán.
- Áp dụng Early Stopping để dừng huấn luyện sớm nếu mô hình không cải thiện.
- Theo dõi loss/accuracy để điều chỉnh learning rate hoặc batch size.
- Chia dữ liệu hợp lý thành tập train/validation/test để đánh giá mô hình chính xác.
- Sử dụng tokenizer phù hợp với mô hình gốc để tránh lỗi khi xử lý dữ liệu đầu vào.

## 6.6. Demo code mô hình LoRA và QLoRA

```
import gradio as gr
import torch
from transformers import AutoModelForSequenceClassification, AutoTokenizer

Khởi tạo mô hình mặc định
MODEL_NAME = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
Biên lưu mô hình đã train và dữ liệu sản phẩm
trained_model = None
product_data = []
Hàm tạo dữ liệu giả cửa hàng giả lập
def generate_data():
 data = [
 ("Giá sản phẩm A là 100,000 VND", 0),
 ("Giá sản phẩm B là 200,000 VND", 1),
 ("Giá sản phẩm C là 150,000 VND", 0)
]
 return data
```

# Hàm train mô hình

```
def train_model():
 global trained_model, product_data
 data = generate_data()
 model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME,
num_labels=2)
 inputs = tokenizer([text for text, _ in data], padding=True, truncation=True,
return_tensors="pt")
 labels = torch.tensor([label for _, label in data])
 outputs = model(**inputs, labels=labels)
 trained_model = model
 # Lưu lại dữ liệu sản phẩm sau khi train
 product_data = {f"Sản phẩm {chr(65+i)}": text.split("là")[1].strip() for i, (text, _) in
enumerate(data)}
 return "Mô hình đã được train."
```

# Hàm hỏi mô hình về giá sản phẩm

```
def ask_model(question):
 if trained_model is None:
 return "Mô hình chưa được train."
 for product, price in product_data.items():
 if product.lower() in question.lower():
 return f"Giá {product} là {price}"
 return "Mô hình không có dữ liệu về sản phẩm này."
```

```
Tạo giao diện Gradio
with gr.Blocks() as demo:
 gr.Markdown("# Demo Train Model và Hỏi Đáp về Giá Sản Phẩm")
 train_button = gr.Button("Train Model")
 question_input = gr.Textbox(label="Câu hỏi về giá sản phẩm")
 ask_button = gr.Button("Hỏi mô hình") # Nút hỏi mô hình riêng biệt
 answer_output = gr.Textbox(label="Trả lời")
 train_button.click(train_model, outputs=[answer_output])
 ask_button.click(ask_model, inputs=[question_input], outputs=[answer_output]) #
 Kết nối nút "Hỏi mô hình" với hàm ask_model
demo.launch()
```

Giải thích code: Khởi tạo mô hình AI và train thêm cho mô hình dữ liệu mà mô hình AI không. Giá sản phẩm của cửa hàng, sau khi train AI sẽ hiểu được và trả lời được giá sản phẩm.

**Kết quả mô hình:** Khi được adapt thêm giá của cửa hàng, mà các ngôn ngữ khác không biết được về dữ liệu của mình, khi train thêm mô hình AI LLM sẽ hiểu được và trả về kết quả giá của sản phẩm

## **Chương 7: Huấn luyện mô hình mã nguồn mở để cạnh tranh với Frontier Models trong dự đoán giá**

### **7.1. Bối cảnh và Mục tiêu**

- Frontier models như GPT-4, Claude 3, Gemini Ultra rất mạnh → nhưng:
  - Chi phí sử dụng API rất cao.
  - Không kiểm soát được dữ liệu đầu vào/ra.
- Open-source LLMs (ví dụ: Mistral, Llama 3, DeepSeek) → nếu Fine-tuning tốt, có thể:
  - Đạt hiệu suất tương đương (hoặc thậm chí tốt hơn trên tác vụ chuyên biệt).
  - Tiết kiệm chi phí cực lớn.
  - Toàn quyền kiểm soát dữ liệu và mô hình.

#### **Mục tiêu tuần này:**

- Chúng ta sẽ tìm hiểu về tất cả các bước cần thiết để tinh chỉnh mô hình Llama 2 với 7 tỷ tham số trên GPU T4. Bạn có tùy chọn sử dụng GPU miễn phí trên Google Colab hoặc Kaggle. Mã chạy trên cả hai nền tảng.
- GPU Colab T4 có 16 GB VRAM giới hạn. Chỉ đủ để lưu trữ trọng số của Llama 2–7b, nghĩa là không thể tinh chỉnh hoàn toàn và chúng ta cần sử dụng các kỹ thuật tinh chỉnh hiệu quả về tham số như LoRA hoặc QLoRA.
- Chúng tôi sẽ sử dụng kỹ thuật QLoRA để tinh chỉnh mô hình ở độ chính xác 4 bit và tối ưu hóa việc sử dụng VRAM. Đối với điều đó, chúng tôi sẽ sử dụng hệ sinh thái Hugging Face của các thư viện LLM: transformers, accelerate, peft, trl, và bitsandbytes.

## 7.2.Các bước tổng thể của dự án

### 7.2.1.Bắt đầu:

Chúng ta sẽ bắt đầu bằng việc cài đặt các thư viện cần thiết.

```
%%capture
```

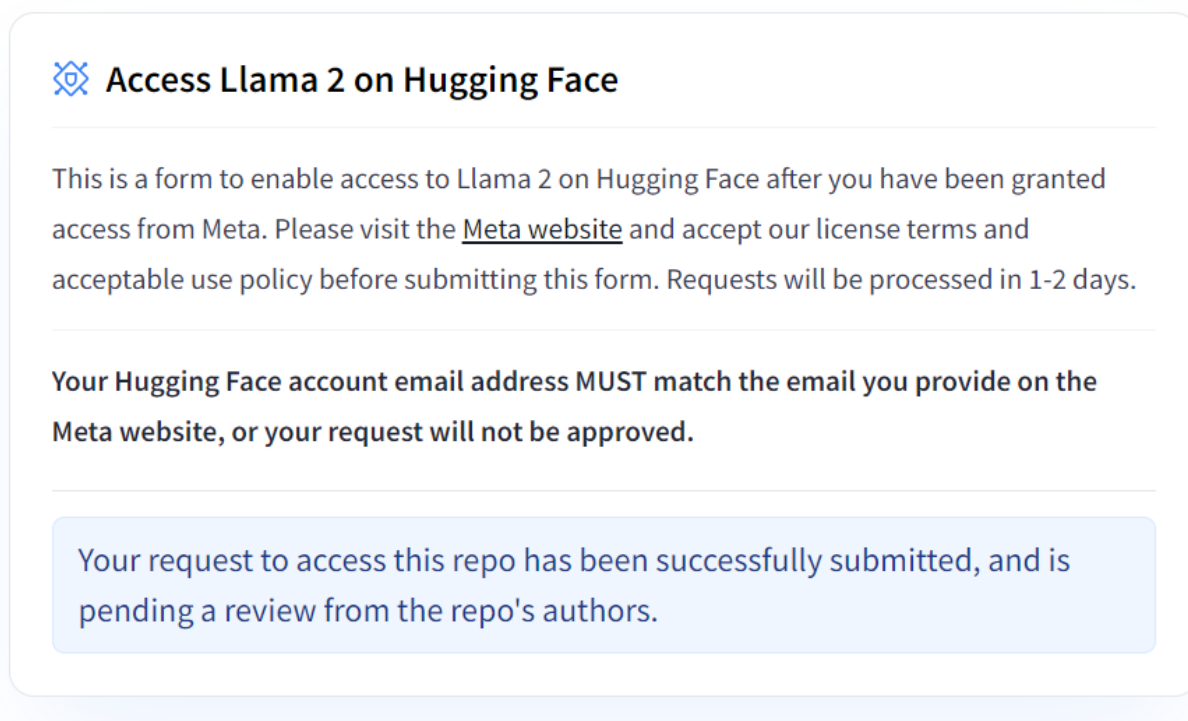
```
%pip install accelerate peft bitsandbytes transformers trl
```

Sau đó, chúng ta sẽ tải các mô-đun cần thiết từ các thư viện này.

```
import os
import torch
from datasets import load_dataset
from transformers import (
 AutoModelForCausalLM,
 AutoTokenizer,
 BitsAndBytesConfig,
 TrainingArguments,
 pipeline,
 logging,
)
from peft import LoraConfig
from trl import SFTTrainer
```

### 7.2.2.Cấu hình mô hình

Truy cập mô hình Llama-2 chính thức của Meta từ Hugging Face, nhưng bạn phải nộp đơn yêu cầu và đợi vài ngày để nhận được xác nhận. Thay vì chờ đợi, chúng tôi sẽ sử dụng Llama-2-7b-chat-hf của NousResearch làm mô hình cơ sở. Nó giống như bản gốc nhưng dễ truy cập.



 **Access Llama 2 on Hugging Face**

This is a form to enable access to Llama 2 on Hugging Face after you have been granted access from Meta. Please visit the [Meta website](#) and accept our license terms and acceptable use policy before submitting this form. Requests will be processed in 1-2 days.

Your Hugging Face account email address **MUST** match the email you provide on the Meta website, or your request will not be approved.

Your request to access this repo has been successfully submitted, and is pending a review from the repo's authors.

*Hình 15: Hình ảnh từ Hugging Face*

Tinh chỉnh mô hình cơ sở của mình bằng cách sử dụng một tập dữ liệu nhỏ hơn có tên là **mlabonne/guanaco-llama2-1k** và viết tên cho mô hình đã tinh chỉnh.

```
Model from Hugging Face hub
base_model = "NousResearch/Llama-2-7b-chat-hf"
New instruction dataset
guanaco_dataset = "mlabonne/guanaco-llama2-1k"
Fine-tuned model
new_model = "llama-2-7b-chat-guanaco"
```



Hình 16: Bộ dữ liệu tại Hugging Face

### 7.2.3. Tải tập dữ liệu, mô hình và bộ mã hóa

Tải tập dữ liệu “guanaco-llama2-1k” từ trung tâm Hugging Face. Tập dữ liệu này chứa 1000 mẫu và đã được xử lý để phù hợp với định dạng nhắc nhở Llama 2, và là một tập hợp con của tập dữ liệu [timdettmers/openassistant-guanaco](#).

```
dataset = load_dataset(guanaco_dataset, split="train")
```

Kết quả:

Dataset parquet downloaded and prepared to

/root/.cache/huggingface/datasets/parquet/mlabonne--guanaco-llama2-1k-

f1f1134768f90029/0.0.0/0b6d5799bb726b24ad7fc7be720c170d8e497f575d02d47537d

e9a5bac074901. Subsequent calls will reuse this data.

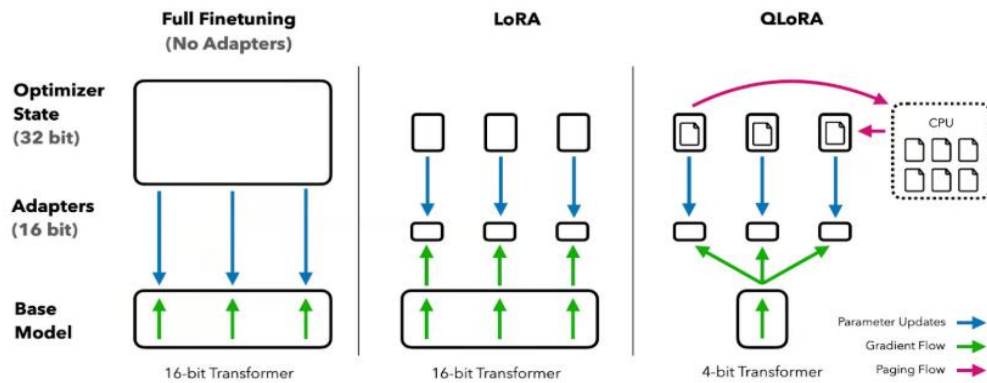


#### 7.2.4. Cấu hình lượng tử hóa 4 bit

Lượng tử hóa 4 bit thông qua QLoRA cho phép tinh chỉnh hiệu quả các mô hình LLM lớn trên phần cứng của người tiêu dùng trong khi vẫn duy trì hiệu suất cao. Điều này cải thiện đáng kể khả năng truy cập và khả năng sử dụng cho các ứng dụng trong thế giới thực.

QLoRA lượng tử hóa mô hình ngôn ngữ được đào tạo trước thành 4 bit và đóng băng các tham số. Sau đó, một số lượng nhỏ các lớp Bộ điều hợp cấp thấp có thể đào tạo được sẽ được thêm vào mô hình.

Trong quá trình tinh chỉnh, các gradient được truyền ngược qua mô hình lượng tử hóa 4 bit đã đóng băng vào chỉ các lớp Bộ điều hợp hạng thấp. Vì vậy, toàn bộ mô hình được đào tạo trước vẫn cố định ở 4 bit trong khi chỉ có các bộ điều hợp được cập nhật. Ngoài ra, lượng tử hóa 4 bit không làm giảm hiệu suất của mô hình.



Hình 17: Full Finetuning, LoRA, và QLoRA

Hình 17 so sánh ba phương pháp huấn luyện mô hình ngôn ngữ: **Full Finetuning**, **LoRA**, và **QLoRA**, tập trung vào cách cập nhật tham số, sử dụng tài nguyên, và luồng gradient. Dưới đây là phần giải thích chi tiết:

##### Full Finetuning (Huấn luyện toàn bộ mô hình)

- Optimizer State (Trạng thái bộ tối ưu hóa): Lưu trên 32-bit.
- Không dùng Adapter, cập nhật tất cả tham số của mô hình gốc.
- Transformer sử dụng độ chính xác 16-bit (FP16).

- Luồng cập nhật tham số (xanh dương): Cập nhật trực tiếp các tham số của mô hình gốc.
- Luồng gradient (xanh lá): Đi xuyên toàn bộ mô hình.
- Tốn tài nguyên rất lớn (RAM & VRAM) vì phải lưu trạng thái tối ưu hóa toàn bộ mô hình và huấn luyện tất cả tham số.

### **LoRA (Low-Rank Adaptation)**

- Sử dụng Adapter 16-bit chèn vào một số tầng của mô hình.
- Chỉ huấn luyện các Adapter, không thay đổi trọng số gốc.
- Transformer vẫn 16-bit, nên mô hình vẫn cần khá nhiều tài nguyên, nhưng ít hơn full finetuning.
- Luồng cập nhật tham số (xanh dương): Chỉ áp dụng vào các Adapter nhỏ.
- Luồng gradient (xanh lá): Chỉ qua các Adapter.
- Không cần lưu trạng thái tối ưu hóa toàn bộ mô hình, giảm tải bộ nhớ.

### **QLoRA (Quantized LoRA)**

- Transformer được lượng tử hóa xuống 4-bit (rất tiết kiệm RAM/VRAM).
- Adapters 16-bit như LoRA.
- Chỉ huấn luyện Adapter, giống LoRA.
- Luồng cập nhật tham số (xanh dương): Chỉ trên Adapter.
- Luồng gradient (xanh lá): Chỉ đi qua Adapter.
- Dữ liệu lượng tử hóa được “paging” lên CPU khi cần thiết (màu hồng), giúp huấn luyện các mô hình rất lớn trên phần cứng giới hạn.
- Cực kỳ tiết kiệm bộ nhớ, thích hợp cho huấn luyện mô hình lớn trên GPU consumer hoặc môi trường giới hạn tài nguyên.

Trong trường hợp này, tạo lượng tử hóa 4 bit với cấu hình kiểu NF4 bằng cách sử dụng BitsAndBytes.

```
compute_dtype = getattr(torch, "float16")
```

```
quant_config = BitsAndBytesConfig(
```

```
 load_in_4bit=True,
```

```
 bnb_4bit_quant_type="nf4",
```

```
bnb_4bit_compute_dtype=compute_dtype,
bnb_4bit_use_double_quant=False)
```

### 7.2.5. Tải mô hình Llama 2

Tải một mô hình sử dụng độ chính xác 4 bit với kiểu dữ liệu tính toán "float16" từ Hugging Face để đào tạo nhanh hơn:

```
model = AutoModelForCausalLM.from_pretrained(
 base_model,
 quantization_config=quant_config,
 device_map={"": 0})
model.config.use_cache = False
model.config.pretraining_tp = 1
```

Tiếp theo, tải trình phân tích cú pháp từ Huggingface và đặt padding\_side thành "right" để khắc phục sự cố với fp16:

```
tokenizer = AutoTokenizer.from_pretrained(base_model, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"
```

Việc tinh chỉnh truyền thống các mô hình ngôn ngữ được đào tạo trước (PLM) đòi hỏi phải cập nhật tất cả các tham số của mô hình, việc này tốn kém về mặt tính toán và cần lượng dữ liệu khổng lồ.

Parameter-Efficient Fine-Tuning (PEFT) hoạt động bằng cách chỉ cập nhật một tập hợp con nhỏ các tham số có ảnh hưởng nhất của mô hình, giúp mô hình hiệu quả hơn nhiều.

```
peft_params = LoraConfig(lora_alpha=16, lora_dropout=0.1, r=64,
 bias="none", task_type="CAUSAL_LM")
```

### 7.2.6. Các thông số đào tạo

Dưới đây là danh sách các siêu tham số có thể được sử dụng để tối ưu hóa quá trình đào tạo:

```
training_params = TrainingArguments(
 output_dir="./results",
 num_train_epochs=1,
 per_device_train_batch_size=4,
 gradient_accumulation_steps=1,
 optim="paged_adamw_32bit",
 save_steps=25,
 logging_steps=25,
 learning_rate=2e-4,
 weight_decay=0.001,
 fp16=False,
 bf16=False,
 max_grad_norm=0.3,
 max_steps=-1,
 warmup_ratio=0.03,
 group_by_length=True,
 lr_scheduler_type="constant",
 report_to="tensorboard"
)
```

- **output\_dir:** Thư mục đầu ra là nơi lưu trữ các dự đoán và điểm kiểm tra của mô hình.
- **num\_train\_epochs:** Một kỷ nguyên đào tạo.
- **fp16/bf16:** Vô hiệu hóa đào tạo fp16/bf16.
- **per\_device\_train\_batch\_size:** Kích thước lô cho mỗi GPU để đào tạo.
- **per\_device\_eval\_batch\_size:** Kích thước lô cho mỗi GPU để đánh giá.
- **gradient\_accumulation\_steps:** Chỉ số này đề cập đến số bước cần thiết để tích lũy các gradient trong quá trình cập nhật.
- **gradient\_checkpointing:** Bật tính năng kiểm tra điểm gradient.
- **max\_grad\_norm:** Cắt màu theo độ dốc.
- **learning\_rate:** Tốc độ học ban đầu.
- **weight\_decay:** Giảm trọng số được áp dụng cho tất cả các lớp ngoại trừ trọng số bias/LayerNorm.
- **Optim:** Trình tối ưu hóa mô hình (AdamW optimizer).
- **lr\_scheduler\_type:** Lịch trình tốc độ học tập.
- **max\_steps:** Số bước đào tạo.
- **warmup\_ratio:** Tỷ lệ các bước khởi động tuyến tính.
- **group\_by\_length:** Điều này có thể cải thiện đáng kể hiệu suất và đẩy nhanh quá trình đào tạo.
- **save\_steps:** Lưu điểm kiểm tra sau mỗi 25 bước cập nhật.
- **logging\_steps:** Ghi nhật ký sau mỗi 25 bước cập nhật.

**Điều chỉnh có giám sát (SFT)** là một bước quan trọng trong việc học tăng cường từ phản hồi của con người (RLHF). Thư viện TRL từ HuggingFace cung cấp một API để sử dụng để tạo các mô hình SFT và đào tạo chúng trên tập dữ liệu của bạn chỉ bằng một vài dòng mã. Nó đi kèm với các công cụ để đào tạo các mô hình ngôn ngữ bằng cách sử dụng học tăng cường, bắt đầu bằng điều chỉnh có giám sát, sau đó là mô hình phần thưởng và cuối cùng là tối ưu hóa chính sách gần (PPO).

Cung cấp cho SFT Trainer mô hình, tập dữ liệu, cấu hình Lora, mã thông báo và các tham số đào tạo.

```
trainer = SFTTrainer(
 model=model,
 train_dataset=dataset,
 peft_config=peft_params,
 dataset_text_field="text",
 max_seq_length=None,
 tokenizer=tokenizer,
 args=training_params,
 packing=False,
)
```

Sử dụng `.train()` để tinh chỉnh mô hình Llama 2 trên một tập dữ liệu mới ta có kết quả tập chạy:

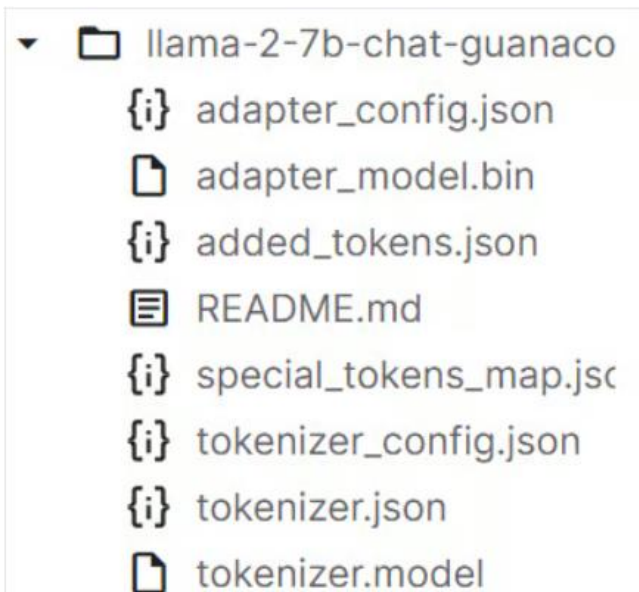
 [250/250 42:50, Epoch 1/1]

| Step | Training Loss |
|------|---------------|
| 25   | 1.346600      |
| 50   | 1.613300      |
| 75   | 1.208800      |
| 100  | 1.437900      |
| 125  | 1.175600      |
| 150  | 1.360300      |
| 175  | 1.171800      |
| 200  | 1.458000      |
| 225  | 1.154300      |
| 250  | 1.529700      |

Sau khi đào tạo mô hình, chúng ta sẽ lưu người áp dụng mô hình và các tokenizer.

Cũng có thể tải mô hình lên Hugging Face bằng API tương tự.

```
trainer.model.save_pretrained(new_model)
trainer.tokenizer.save_pretrained(new_model)
```



### 7.2.7. Đánh giá

Xem lại kết quả đào tạo trong phiên tương tác của Tensorboard.:

```
from tensorboard import notebook
log_dir = "results/runs"
notebook.start("--logdir {} --port 4000".format(log_dir))
```

Kết quả ta được biểu đồ như sau:



- ❖ Để kiểm tra mô hình tinh chỉnh của mình, chúng tôi sẽ sử dụng transformers quy trình tạo văn bản và đặt những câu hỏi đơn giản như " Who is Leonardo Da Vinci?".

```
logging.set_verbosity(logging.CRITICAL)
prompt = "Who is Leonardo Da Vinci?"
pipe = pipeline(task="text-generation", model=model, tokenizer=tokenizer,
max_length=200)
result = pipe(f"<s>[INST] {prompt} [/INST]")
print(result[0]['generated_text'])
```

**Đầu ra ta có kết quả như sau:**

<s>[INST] Who is Leonardo Da Vinci? [/INST] Leonardo da Vinci (1452-1519) was an Italian polymath, artist, inventor, and engineer.

Da Vinci is widely considered one of the greatest painters of all time, and his works include the famous Mona Lisa. He was also an accomplished engineer, inventor, and anatomist, and his designs for machines and flight were centuries ahead of his time.

Da Vinci was born in the town of Vinci, Italy, and he was the illegitimate son of a local notary. Despite his humble origins, he was able to study art and engineering in Florence, and he became a renowned artist and inventor.

Da Vinci's work had a profound impact on the Renaissance, and his legacy continues to inspire artists, engineers, and inventors to this day. He

```
<s>[INST] Who is Leonardo Da Vinci? [/INST] Leonardo da Vinci (1452-1519) was an Italian polymath, artist, inventor, and engineer.
Da Vinci is widely considered one of the greatest painters of all time, and his works include the famous Mona Lisa. He was also an accomplished engineer, inventor, and anatomist, and his
Da Vinci was born in the town of Vinci, Italy, and he was the illegitimate son of a local notary. Despite his humble origins, he was able to study art and engineering in Florence, and he
Da Vinci's work had a profound impact on the Renaissance, and his legacy continues to inspire artists, engineers, and inventors to this day. He
```



❖ Ta sẽ thử hỏi một câu hỏi khác: “What is Datacamp Career track?”

```
prompt = "What is Datacamp Career track?"
result = pipe(f"<s>[INST] {prompt} [/INST]"
print(result[0]['generated_text'])
```

**Đầu ra ta có kết quả như sau:**

<s>[INST] What is Datacamp Career track? [/INST] DataCamp Career Track is a program that offers a comprehensive learning experience to help you build your skills and prepare for a career in data science.

The program includes a range of courses, projects, and assessments that are designed to help you build your skills in data science. You will learn how to work with data, create visualizations, and build predictive models.

In addition to the technical skills, you will also learn how to communicate your findings to stakeholders and how to work with a team to solve complex problems.

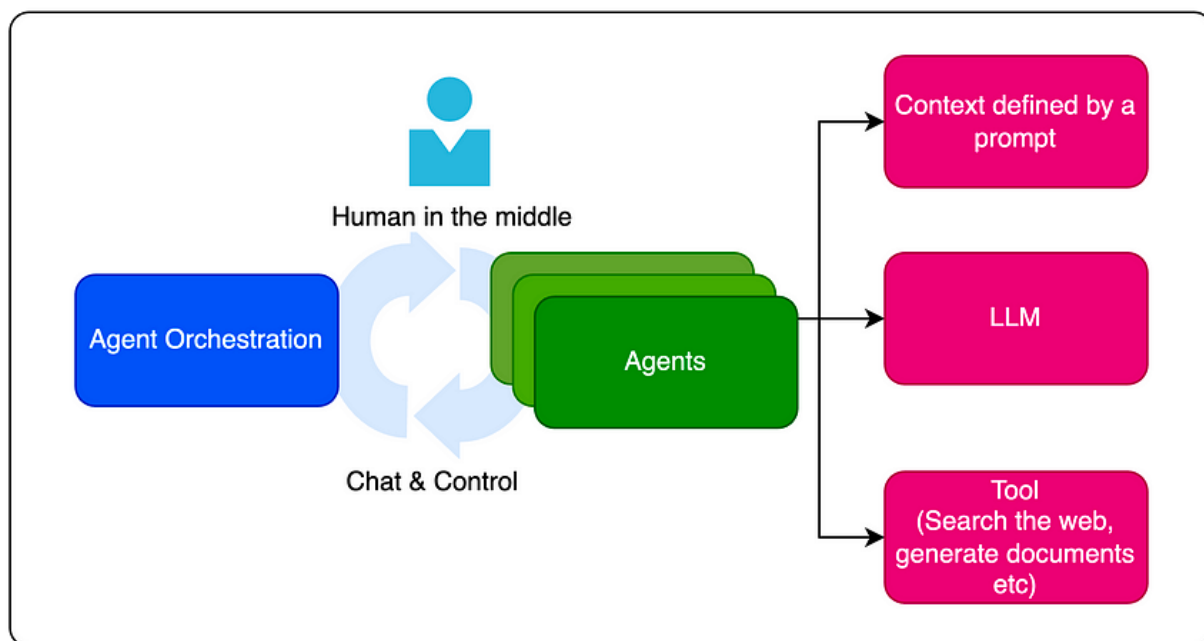
The program is designed to be flexible, so you can learn at your own pace and on your own schedule. You will also have access to a community of learners and mentors who can provide support and guidance throughout the program.

Overall, DataCamp Career Track is a great way to build your skills and prepare for a career in

```
<s>[INST] What is Datacamp Career track? [/INST] DataCamp Career Track is a program that offers a comprehensive learning experience to help you build your skill
The program includes a range of courses, projects, and assessments that are designed to help you build your skills in data science. You will learn how to work w
In addition to the technical skills, you will also learn how to communicate your findings to stakeholders and how to work with a team to solve complex problems.
The program is designed to be flexible, so you can learn at your own pace and on your own schedule. You will also have access to a community of learners and men
Overall, DataCamp Career Track is a great way to build your skills and prepare for a career in
```

## Chương 8: Xây dựng Hệ thống Tự động hóa với Các tác nhân (Agents) Hợp tác cùng các Mô hình LLM

### 8.1. Giới thiệu về Multi-Agent Systems (MAS)



Hình 18: Mô hình Multi-Agent Systems (MAS)

#### Hình 18 mô tả hệ thống MAS:

**Multi-Agent System (MAS)** là hệ thống gồm nhiều tác nhân (agents) có thể hoạt động độc lập hoặc hợp tác để đạt được một mục tiêu chung.

- Tác nhân có thể là phần mềm hoặc robot có khả năng tương tác, ra quyết định, và phối hợp.
- Trong ngữ cảnh AI, các tác nhân thường là các mô hình LLMs hoặc API có thể giao tiếp với nhau để giải quyết vấn đề.

### **8.1.1 Agent Orchestration (Điều phối tác nhân):**

- Là trung tâm điều khiển hệ thống.
- Điều phối các tác nhân (Agents) để xử lý các nhiệm vụ.
- Kết nối người dùng và các tác nhân trong hệ thống.

### **8.1.2 Agents (Tác nhân):**

- Là các đơn vị xử lý độc lập có thể thực hiện các nhiệm vụ riêng biệt.
- Có thể bao gồm nhiều tác nhân cùng hoạt động song song.
- Các tác nhân này có thể tương tác với:
  - Context (Ngữ cảnh): Được xác định thông qua prompt (yêu cầu hoặc câu hỏi).
  - LLM (Large Language Model - Mô hình ngôn ngữ lớn): Như GPT, sử dụng để xử lý và trả lời câu hỏi.
  - Tools (Công cụ): Bao gồm các chức năng như tìm kiếm web, sinh tài liệu, phân tích dữ liệu.

### **8.1.3 Human in the middle (Người dùng ở giữa):**

Người dùng là trung tâm của hệ thống:

- Đưa ra câu hỏi hoặc yêu cầu.
- Giám sát và điều khiển hoạt động của các tác nhân.
- Có thể can thiệp vào quy trình khi cần.

### **8.1.4. Quy trình hoạt động:**

- Người dùng gửi câu hỏi hoặc yêu cầu đến hệ thống.
- Agent Orchestration nhận yêu cầu, phân phối đến các tác nhân phù hợp.
- Mỗi tác nhân sử dụng ngữ cảnh, mô hình ngôn ngữ (LLM) và công cụ để xử lý.
- Kết quả được trả về cho người dùng.
- Người dùng có thể phản hồi hoặc điều chỉnh tương tác với tác nhân.

## **8.2. Tại sao cần Multi-Agent Systems?**

- Đơn giản hóa các tác vụ phức tạp: Một tác vụ đơn lẻ có thể được chia thành nhiều bước (subtasks), mỗi bước do một tác nhân thực hiện.
- Tăng hiệu quả: Các tác nhân có thể hợp tác để hoàn thành công việc nhanh hơn và chính xác hơn.
- Khả năng mở rộng: Thêm tác nhân mới vào hệ thống để mở rộng các tác vụ mà không làm giảm hiệu suất của hệ thống.

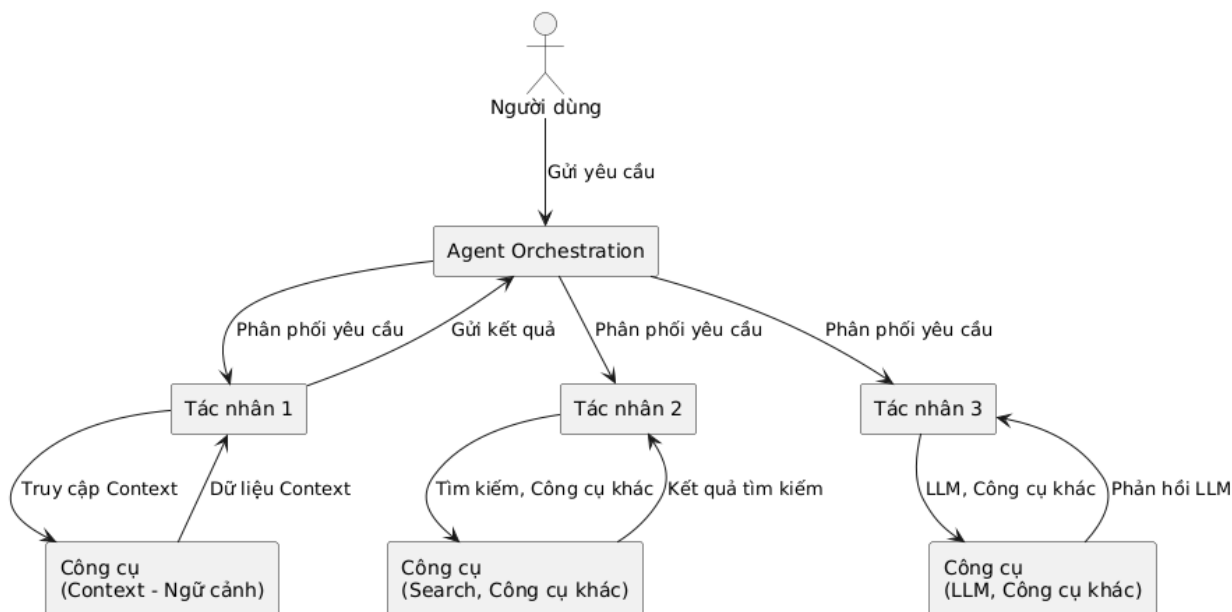
## **8.3. Các loại Tác nhân trong MAS**

- Reactive Agents: Đáp ứng trực tiếp với môi trường hoặc yêu cầu mà không có chiến lược dài hạn (ví dụ: một chatbot đơn giản trả lời câu hỏi).
- Deliberative Agents: Phân tích và ra quyết định trước khi hành động (ví dụ: một tác nhân đánh giá chiến lược đầu tư).
- Collaborative Agents: Làm việc cùng nhau để giải quyết vấn đề phức tạp (ví dụ: một nhóm tác nhân tư vấn tài chính hợp tác để đưa ra một dự báo chính xác).

## 8.4. Xây dựng Hệ thống Tác nhân Tự động (Autonomous Multi-Agent System)

### 8.4.1 Kiến trúc chung

Mô hình hóa luồng yêu cầu Người dùng → Agent Orchestration → Các Tác nhân → Công cụ → Trả kết quả



Hình 19: Kiến trúc hệ thống Tác nhân Tự động (Autonomous Multi-Agent System)

### 8.4.2 Mô hình Collaborative Multi-Agent System

- **Coordinator Agent** (Agent chính):
  - Nhận yêu cầu từ người dùng.
  - Phân tích và chia nhỏ tác vụ thành các sub-tasks.
  - Phân công cho các **Sub-Agent** phù hợp (các mô hình LLM hoặc dịch vụ API).
  - Hợp tác các sub-agent để trả lời hoặc hoàn thành yêu cầu.
- **Sub-Agent**:
  - Mỗi sub-agent là một mô hình LLM (ví dụ: GPT, Claude, Llama, v.v...) có nhiệm vụ thực hiện một phần công việc (tìm kiếm thông tin, phân tích văn bản, trả lời câu hỏi chuyên môn).
  - Tác nhân có thể giao tiếp với nhau (thông qua API hoặc message passing) để phối hợp giải quyết yêu cầu.

## 8.5. Ứng dụng thực tế của Multi-Agent Systems

- **Customer Support:** Một hệ thống hỗ trợ khách hàng có thể sử dụng nhiều tác nhân hợp tác để:
  - Một tác nhân xử lý thông tin đơn giản (ví dụ: kiểm tra đơn hàng).
  - Một tác nhân khác trả lời câu hỏi chuyên môn (ví dụ: kỹ thuật viên sửa chữa).
- **Data Analysis:** Một hệ thống phân tích dữ liệu có thể sử dụng các tác nhân để:
  - Một tác nhân thu thập dữ liệu từ nhiều nguồn khác nhau.
  - Một tác nhân phân tích và xây dựng mô hình dự báo.
- **Recommendation Systems:** Một hệ thống đề xuất có thể sử dụng nhiều tác nhân để:
  - Một tác nhân lấy dữ liệu người dùng (thông tin profile).
  - Một tác nhân khác thực hiện phân tích sở thích và gợi ý sản phẩm.

## 8.6. Công cụ và Frameworks hỗ trợ

- **LangChain:**
  - **LangChain** cung cấp một framework mạnh mẽ để tạo ra các tác nhân (agents) và các workflow hợp tác giữa chúng.
  - **AgentExecutor** trong LangChain giúp quản lý các tác nhân và phân công công việc.
- **AutoGPT:**
  - Một dự án mã nguồn mở cho phép các tác nhân tự động hóa các tác vụ bằng cách sử dụng GPT-4 để giao tiếp với nhau và ra quyết định.
- **Haystack:**
  - Haystack của deepset là một framework khác hỗ trợ xây dựng các hệ thống tìm kiếm thông minh, tích hợp nhiều tác nhân để tìm kiếm và trả lời các câu hỏi.

## 7. Thách thức khi xây dựng Multi-Agent Systems

- **Quản lý và đồng bộ hóa:** Khi có nhiều tác nhân hoạt động cùng một lúc, cần phải có một cơ chế tốt để quản lý các tác vụ và đồng bộ hóa kết quả.
- **Tối ưu hóa chi phí:** Các hệ thống có thể tốn kém chi phí nếu không tối ưu số lượng tác nhân và tài nguyên.

- **Hợp tác hiệu quả:** Các tác nhân phải biết cách giao tiếp với nhau để tránh làm việc trùng lặp hoặc thiếu sót.

#### a.Code Demo

```
Tải khóa API từ file .env
load_dotenv()
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
GOOGLE_SEARCH_API_KEY = os.getenv("GOOGLE_SEARCH_API_KEY")
GOOGLE_SEARCH_CX = os.getenv("GOOGLE_SEARCH_CX")

Kiểm tra và xác thực các khóa API
if not all([GEMINI_API_KEY, GOOGLE_SEARCH_API_KEY,
GOOGLE_SEARCH_CX]):
 raise ValueError("⚠️ Thiếu khóa API trong file .env!")

Cấu hình API Gemini
genai.configure(api_key=GEMINI_API_KEY)

class TacNhan:
 def __init__(self, ten):
 """
 Khởi tạo một tác nhân với tên được chỉ định

 :param ten: Tên của tác nhân
 """
 self.ten = ten

 def xu_ly(self, yeu_cau_nguoi_dung):
 """
 Xử lý yêu cầu của người dùng dựa trên loại tác nhân

 :param yeu_cau_nguoi_dung: Câu hỏi hoặc yêu cầu của người dùng
 :return: Kết quả xử lý từ tác nhân
 """
 try:
 if self.ten == "Gemini":
 return self.hoi_gemini(yeu_cau_nguoi_dung)
 elif self.ten == "TimKiem":
```

```

 return self.tim_kiem_google(yeu_cau_nguoi_dung)
 else:
 return f"Tác nhân {self.ten}: Không có phương thức xử lý."
except Exception as e:
 return f"Tác nhân {self.ten}: Lỗi khi xử lý yêu cầu. Chi tiết: {str(e)}"

def hoi_gemini(self, van_ban):
 """
 Gọi API Gemini để trả lời câu hỏi

 :param van_ban: Văn bản câu hỏi
 :return: Phản hồi từ Gemini
 """
 try:
 # Chức năng để in ra danh sách model
 def in_danh_sach_model():
 print("Danh sách model Gemini:")
 models = genai.list_models()
 for model in models:
 try:
 print(f"Model: {model.name}")
 except Exception as e:
 print(f"Lỗi khi in model: {e}")

 # In danh sách model
 in_danh_sach_model()

 # Thử nghiệm với các model phổ biến
 cac_model_thu = [
 'gemini-1.5-pro-latest',
 'gemini-pro',
 'gemini-1.0-pro',
 'gemini-1.5-flash-latest'
]

```



```

Thử từng model
for model_name in cac_model_thu:
 try:
 # Tạo model
 model = genai.GenerativeModel(model_name)

 # Cấu hình sinh văn bản
 cau_hinh_sinh = {
 "temperature": 0.7, # Độ sáng tạo của câu trả lời
 "max_output_tokens": 2048, # Số token tối đa cho câu trả lời
 }

 # Sinh nội dung
 phan_hoi = model.generate_content(
 van_ban,
 generation_config=cau_hinh_sinh
)

 return f"Tác nhân Gemini: {phan_hoi.text.strip()}"

 except Exception as model_error:
 print(f"Lỗi với model {model_name}: {model_error}")

Nếu không model nào hoạt động
raise ValueError("Không tìm thấy model Gemini phù hợp")

except Exception as e:
 return f"Tác nhân Gemini: Lỗi gọi API. Chi tiết: {str(e)}"

def tim_kiem_google(self, truy_van):
 """
 Gửi yêu cầu tìm kiếm đến Google Custom Search API

 :param truy_van: Từ khóa tìm kiếm
 :return: Kết quả tìm kiếm
 """

```

```

try:
 url = "https://www.googleapis.com/customsearch/v1"
 tham_so = {
 "q": truy_van,
 "key": GOOGLE_SEARCH_API_KEY,
 "cx": GOOGLE_SEARCH_CX
 }
 phan_hoi = requests.get(url, params=tham_so, timeout=10)

 if phan_hoi.status_code == 200:
 du_lieu = phan_hoi.json()
 if 'items' in du_lieu:
 ket_qua = du_lieu['items'][0]['snippet']
 return f"Tác nhân Tìm kiếm: Kết quả tìm kiếm: {ket_qua}"
 else:
 return "Tác nhân Tìm kiếm: Không tìm thấy kết quả."
 else:
 return f"Tác nhân Tìm kiếm: Lỗi tìm kiếm. Mã trạng thái:
{phan_hoi.status_code}"
 except requests.RequestException as e:
 return f"Tác nhân Tìm kiếm: Lỗi mạng. Chi tiết: {str(e)}"

class DieuPhoi:
 def __init__(self):
 """
 Khởi tạo hệ thống điều phối các tác nhân
 """
 self.cac_tac_nhan = []

 def them_tac_nhan(self, tac_nhan):
 """
 Thêm tác nhân vào hệ thống

 :param tac_nhan: Tác nhân cần thêm
 """
 self.cac_tac_nhan.append(tac_nhan)

```

```

def phan_phoi_nhiem_vu(self, yeu_cau_nguoi_dung):
 """
 Phân phối nhiệm vụ đến tất cả các tác nhân và thu thập kết quả
 :param yeu_cau_nguoi_dung: Yêu cầu của người dùng
 :return: Danh sách kết quả từ các tác nhân
 """

 ket_qua = []
 for tac_nhan in self.cac_tac_nhan:
 ket_qua_tac_nhan = tac_nhan.xu_ly(yeu_cau_nguoi_dung)
 ket_qua.append((tac_nhan.ten, ket_qua_tac_nhan))
 return ket_qua

class GiaoDienNguoiDung:
 def __init__(self, man_hinh_chinh):
 """
 Khởi tạo giao diện người dùng
 :param man_hinh_chinh: Cửa sổ giao diện chính
 """

 self.man_hinh_chinh = man_hinh_chinh
 man_hinh_chinh.title("Hệ Thống Tác Nhân Thông Minh")
 man_hinh_chinh.geometry("500x600")
 # Tạo hệ thống điều phối và thêm tác nhân
 self.dieu_phoi = DieuPhoi()
 self.dieu_phoi.them_tac_nhan(TacNhan("Gemini"))
 self.dieu_phoi.them_tac_nhan(TacNhan("TimKiem"))
 # Tạo và thiết lập các thành phần giao diện
 self.tao_cac_phan_tu()

 def tao_cac_phan_tu(self):
 # Nhấn hướng dẫn nhập câu hỏi
 tk.Label(self.man_hinh_chinh, text="Nhập câu hỏi của bạn:").pack(pady=10)
 # Ô nhập câu hỏi
 self.o_nhap = tk.Entry(self.man_hinh_chinh, width=60)
 self.o_nhap.pack(pady=10)
 self.o_nhap.bind('<Return>', self.hoi_cau_hoi) # Hỗ trợ phím Enter
 # Nút "Hỏi"
 nut_hoi = tk.Button(self.man_hinh_chinh, text="Hỏi",

```

```

command=self.hoi_cau_hoi)
 nut_hoi.pack(pady=10)
 # Vùng hiển thị kết quả
 self.vung_ket_qua = tk.Text(self.man_hinh_chinh, height=20, width=60,
wrap=tk.WORD)
 self.vung_ket_qua.pack(pady=10)
 # Thanh cuộn cho vùng kết quả
 thanh_cuon = tk.Scrollbar(self.man_hinh_chinh,
command=self.vung_ket_qua.yview)
 thanh_cuon.pack(side=tk.RIGHT, fill=tk.Y)
 self.vung_ket_qua.config(yscrollcommand=thanh_cuon.set)
def hoi_cau_hoi(self, su_kien=None):
 """
 Xử lý câu hỏi của người dùng và hiển thị kết quả
 :param su_kien: Sự kiện (được sử dụng khi nhấn Enter)
 """
 cau_hoi = self.o_nhap.get().strip()
 if cau_hoi:
 # Xóa kết quả cũ
 self.vung_ket_qua.delete(1.0, tk.END)
 # Phân phối nhiệm vụ và hiển thị kết quả
 ket_qua = self.dieu_phoi.phan_phoi_nhiem_vu(cau_hoi)
 for ten_tac_nhan, noi_dung in ket_qua:
 self.vung_ket_qua.insert(tk.END, f"{noi_dung}\n\n")

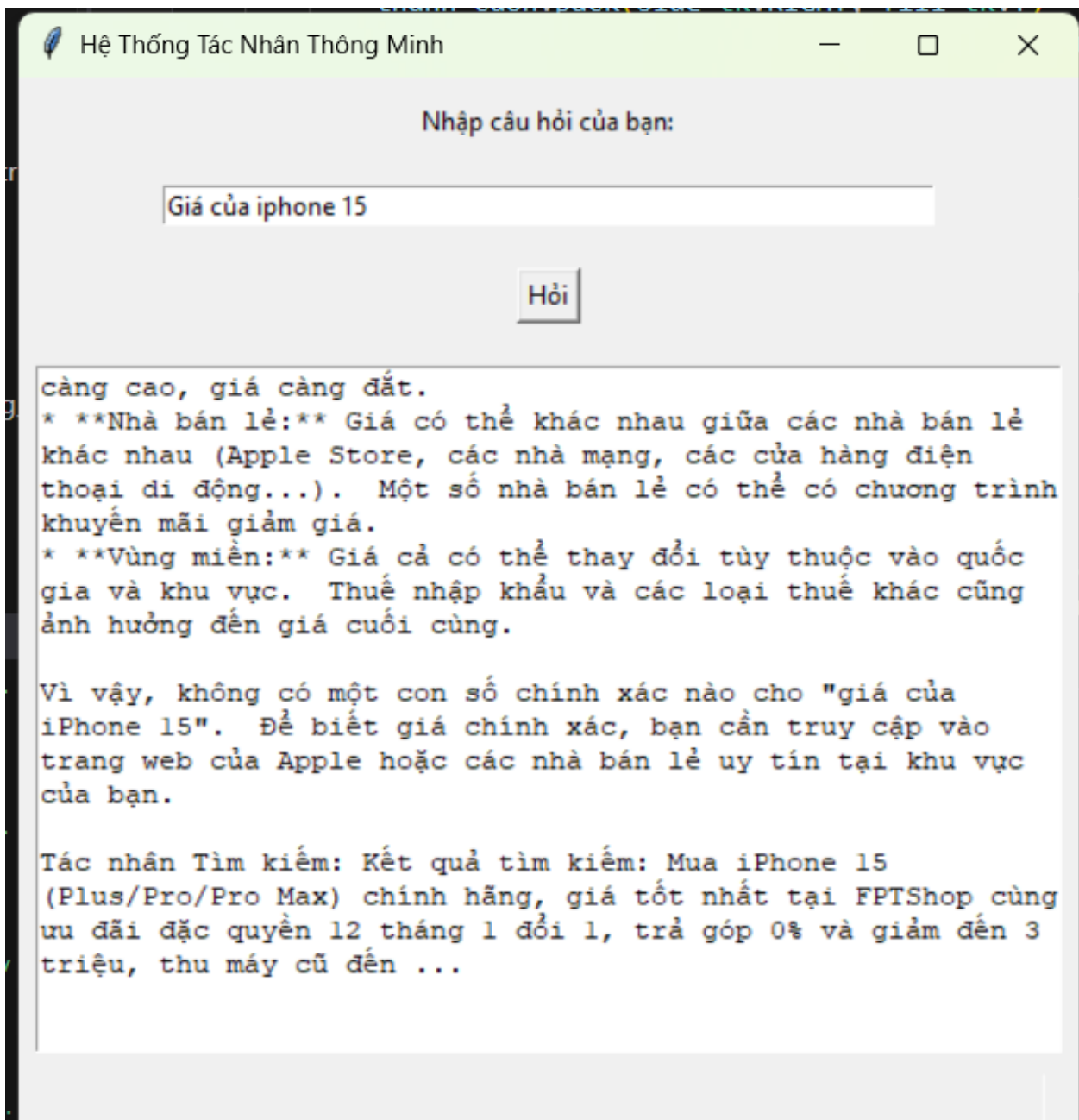
 # Xóa ô nhập
 self.o_nhap.delete(0, tk.END)
def chinh(danh_sach_tham_so=None):
 """
 Hàm chính để khởi chạy ứng dụng
 """
 man_hinh = tk.Tk()
 ung_dung = GiaoDienNguoiDung(man_hinh)
 man_hinh.mainloop()

if __name__ == "__main__":
 chinh()

```

**Giải thích code:** kết hợp hai Agent dùng gemini để trả lời câu hỏi, dùng Agent tool tìm kiếm bằng Google Custom Search API search đến trang web FPT.

Kết quả sẽ trả về như sau



Hệ Thống Tác Nhân Thông Minh

Nhập câu hỏi của bạn:

Giá của iphone 15

Hỏi

càng cao, giá càng đắt.

\* \*\*Nhà bán lẻ:\*\* Giá có thể khác nhau giữa các nhà bán lẻ khác nhau (Apple Store, các nhà mạng, các cửa hàng điện thoại di động...). Một số nhà bán lẻ có thể có chương trình khuyến mãi giảm giá.

\* \*\*Vùng miền:\*\* Giá cả có thể thay đổi tùy thuộc vào quốc gia và khu vực. Thuế nhập khẩu và các loại thuế khác cũng ảnh hưởng đến giá cuối cùng.

Vì vậy, không có một con số chính xác nào cho "giá của iPhone 15". Để biết giá chính xác, bạn cần truy cập vào trang web của Apple hoặc các nhà bán lẻ uy tín tại khu vực của bạn.

Tác nhân Tìm kiếm: Kết quả tìm kiếm: Mua iPhone 15 (Plus/Pro/Pro Max) chính hãng, giá tốt nhất tại FPTShop cùng ưu đãi đặc quyền 12 tháng 1 đổi 1, trả góp 0% và giảm đến 3 triệu, thu máy cũ đến ...

## Chương 9: Tóm tắt nội dung học được

Sau khi hoàn thành 8 chương học này, tụi em đã hiểu và học được:

1. Xây dựng và triển khai sản phẩm AI hoàn chỉnh dựa trên LLMs (Large Language Models).
2. Tạo và tùy chỉnh giao diện người dùng bằng Gradio, giúp tương tác dễ dàng với mô hình AI.
3. Sử dụng Hugging Face để xây dựng các giải pháp AI nguồn mở, bao gồm tạo mô hình tùy chỉnh và tự động hóa tác vụ.
4. Đánh giá và lựa chọn mô hình LLM phù hợp cho các tác vụ khác nhau như sinh mã (code generation) và hỗ trợ nghiệp vụ.
5. Xây dựng các giải pháp RAG (Retrieval-Augmented Generation) với Vector Embeddings và LangChain, tăng cường khả năng tìm kiếm và xử lý thông tin.
6. Tinh chỉnh mô hình LLM tiên tiến bằng kỹ thuật LoRA/QLoRA, giúp tối ưu hiệu suất mà không cần tài nguyên lớn.
7. Xây dựng hệ thống Multi-Agent Systems (MAS), nơi các tác nhân AI có thể hợp tác hoặc hoạt động độc lập để hoàn thành mục tiêu chung.